UNIVERSITÉ LIBRE DE BRUXELLES

BRUFACE BRUSSELS FACULTY OF ENGINEERING

VRIJE UNIVERSITEIT BRUSSEL

Master thesis

Perception and control of a robotic manipulator for pushing of objects

Author :

KEISER Antoine

Promotors :

prof. dr. ir. Greet van de Perre

ir. Mohayad Omer

2025-2026

Abstract

Non-prehensile pushing offers an attractive alternative to grasping for objects that are too large, heavy, or awkwardly shaped to pick up, but it is difficult to control reliably because the pushed object is never rigidly constrained and its motion depends on contact location, friction, and object inertia. Reactive pushing controllers infer this contact information from a contact-force measurement at the end-effector, typically provided by a wrist-mounted force/torque sensor or by a tactile fingertip.

This thesis proposes and evaluates a reactive pushing controller for a 7-DoF manipulator equipped with a spherical end-effector and a contact-force sensing capability at that end-effector, modelled in simulation by the physics engine's contact solver. The measured contact-force direction drives an impedance-based control law, and a geometric estimate derived from the pusher-object penetration acts as a fallback over the intervals where the sensing channel reports no contact. The controller is structured as a finite-state machine that separates the approach, pushing, and repositioning phases of a push, and is augmented with corrective feedback terms addressing lateral drift and directional oscillation, a camera-based perception model with a dead-reckoning fallback for handling occlusion during contact, and a safety mechanism that detects and recovers from stalled or regressing pushes.

The approach is validated in a physics-based simulation environment across a range of object shapes, masses, friction conditions, and initial contact geometries. An ablation study is used to isolate the contribution of each corrective term, revealing that their usefulness depends strongly on object geometry rather than being uniformly beneficial. Targeted experiments further show that the controller tolerates the loss of up to 90% of visual observations with no degradation of the pushed trajectory, the residual error being concentrated in the stopping decision rather than in the push itself, and quantify the limit case in which the object is never re-observed.

Taken together, these results suggest that a reactive impedance controller driven by end-effector contact-force feedback transfers from the omnidirectional mobile base of prior work to an arm-mounted spherical pusher, provided the controller's corrective terms are adapted to the geometry of the object being pushed.

Contents

List of Figures

List of Tables

1 Introduction

This chapter introduces the motivation behind this thesis and a guideline.

1.1 Motivation

Robotic manipulation systems have achieved significant advances in recent years, driven by progress in sensing, planning, and machine learning, with the demand across various industries increasing, as their efficiency and speed in accomplishing tasks are undeniable. Robots are useful to accomplish complex or repeatable tasks by grasping or even pushing the objects.

Despite significant progress in robotic manipulation, enabling robots to autonomously interact with objects in unstructured environments remains a challenging problem. Non-prehensile manipulation tasks, such as object pushing, require a continuous understanding of the object state and a tight integration between perception, planning, and control. While sensors or camera provide information about the environment, they are inherently noisy and subject to occlusions, making the state estimation difficult. At the same time, physical interaction with objects such as friction can not be neglected and introduces uncertainties that complicate both planning and control of the robot.

Pushing non-prehensile objects in a controlled and autonomous manner is nevertheless a fundamental skill for robotic systems operating in real world scenarios, such as domestic assistance, industrial manipulation, or logistics. This thesis is motivated by the need to develop integrated robotic systems that can perceive their environment, reason about manipulation tasks, and execute contact-rich actions robustly. By focusing on object pushing as a representative manipulation problem, the motivation of this thesis is to contribute and improve toward more autonomous and adaptable robotic non-prehensile manipulation capabilities .

1.2 Thesis outline

This thesis is organised as follows.

Chapter 2 states the problem addressed and the specific objectives pursued.

Chapter 3 reviews the literature relevant to non-prehensile manipulation, the mechanics of planar pushing, planning and control strategies, force-feedback and compliant control, and perception for manipulation.

Chapter 4 develops the theoretical background: the dynamics of the robot and of the pushed object, the grasp matrix relating contact force to object motion, and the contact model relating the measured contact force to the object's planar motion

Chapter 5 presents the proposed control architecture, including the finite-state controller, the impedance-based push law, the camera perception model, and the dead-reckoning and stall/regression safety mechanisms.

Chapter 6 validates the approach in the Drake simulation environment [1], reporting benchmark results across object shape, mass, friction, and initial obliquity, an ablation study isolating the contribution of each controller component, and a robustness evaluation under simulated camera noise and occlusion.

Chapter 7 concludes the thesis, summarising its contributions, limitations, and directions for future work.

2 Objectives

This work aims to provide an efficient and robust perception and control of a robotic arm to achieve a continuous and stable pushing of an object, depending on the shape and properties of this object. The three objectives of this thesis are described below.

Determine where in the geometry of the object to push, to achieve the desired position. The mechanics of planar pushing show that the motion an object undergoes is not determined by the applied force alone, but by where on its boundary that force is applied relative to its center of mass. A contact aligned with the direction of the goal drives the object mostly forward. If there is a contact offset from that direction, even slightly, introduces a torque and an unwanted rotation. Answering this objective means, first, choosing which side of the object to approach and along which direction to push, given its current pose and the goal, and second, keeping the effective contact point aligned with that choice throughout the push, since the sphere and the object both move and small misalignments accumulate rather than cancel out. This objective is shape-dependent: a cube offers a discrete set of flat faces to choose from, while a cylinder's circular boundary offers a continuous radial contact.

Command the end-effector to achieve the desired velocity. Once a desired pushing direction is known, it has to be turned into an actual motion of the robot. This means expressing the intended push as a Cartesian velocity for the end-effector, then mapping that velocity through the manipulator Jacobian into joint torques, so that the command is not just correct in the abstract planar model but also physically realisable on the Franka Panda robot shown in Figure 2.1.

Regulate the command and confirming contact. The applied command has to be kept consistent with the physical reality of the interaction. Regulating the command has two components. First, the push direction should be corrected using the measured contact force, following a reactive force-feedback law, so that the end-effector remains engaged with the object. Secondly, the magnitude of the commanded velocity should be regulated through admittance saturation as a function of the measured contact force, combined with a distance-dependent braking profile that reduces the commanded speed smoothly as the object ap-

proaches the target.

Confirming contact relies on two independent sources, both can be evaluated on the object's real, physical position. The simulator's own contact-force computation, treated as the primary source of truth, and a geometric estimate derived from the measured penetration between the end-effector and the object's known surface, used as a fallback whenever the primary source is unavailable.

Figure 2.1: Franka Emika Panda seven-axis robotic arm [2].

3 State of the art

This chapter reviews the literature relevant to perceiving and controlling a robotic manipulator for the pushing of objects. Robotic manipulation generally relies on three capabilities: perception, which estimates the state of the robot and of the objects it interacts with; planning, which decides which actions to take given that state and a goal; and control, which executes those actions while respecting the dynamics and constraints of the system.

3.1 Non-prehensile manipulation

Manipulation tasks are further commonly divided into prehensile actions, where the object is constrained by a grasp, as shown in Figure 3.1, and non-prehensile actions, where the object is moved without ever being firmly held, as is the case for pushing. In contrast with classical pick-and-place operations, where the object remains constrained by the gripper, non-prehensile actions such as pushing allow the robot to influence object motion beyond the immediate grasping workspace. This makes them attractive to move heavy or large objects that would be hard or even impossible to grab.

These advantages come at the cost of increased complexity. The outcome of an action depends on contact geometry, friction, object inertia, and the evolution of intermittent contact because the object is not rigidly constrained by the manipulator. As a result, perception, planning, and control are tightly coupled in non-prehensile manipulation.

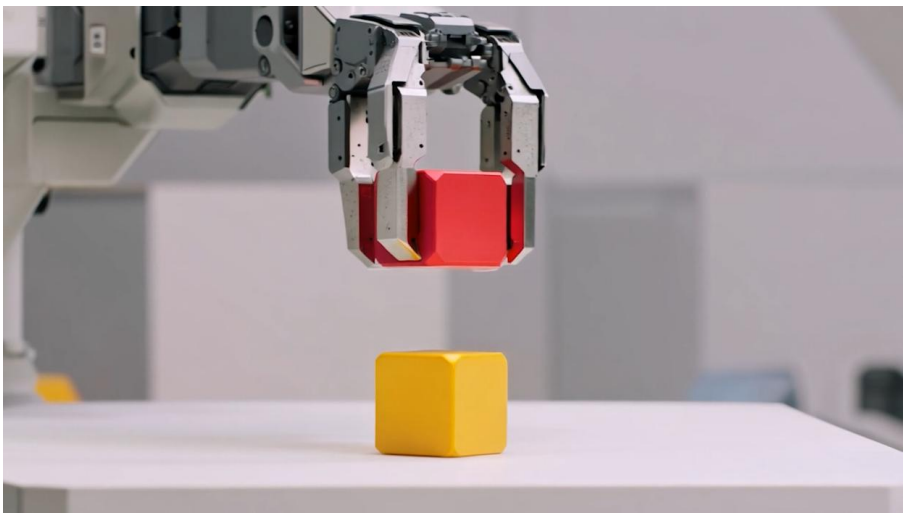


Figure 3.1: Prehensile manipulation (grasping) [3]

3.2 Mechanics of planar pushing

The mechanics of planar pushing provide the theoretical basis for much of the literature on non-prehensile manipulation. Pushing cannot be understood as a purely kinematic problem as the object motion depends on the location of contact, the direction of the applied force, and the distribution of friction at the support surface. [4] established the importance of reasoning directly about contact mechanics when planning pushes.

Lynch and Mason [5] later extended this framework to stable pushing, showing that a pusher can maintain contact with a moving object under appropriate geometric and friction conditions. Their results are particularly interesting because they link the curvature of the object boundary at the contact point to the feasibility of maintaining a stable push.

A complementary perspective was provided by [6], who introduced the limit-surface model. This model maps the wrench applied to an object to its resulting planar motion through the support friction law. Although exact computation of the limit surface requires assumptions on pressure distribution and mass properties, the framework remains central to analytic models of pushing.

Together, these works define the core mechanical insight behind planar pushing: contact forces induce both translation and rotation, and small changes in contact location or friction can significantly alter the object trajectory.

3.3 Planning and control for pushing

Building on these mechanical models, several planning and control strategies have been proposed for goal-directed pushing.

3.3.1 Model-based planning and optimisation

Model-based methods explicitly encode rigid-body dynamics, friction, and contact constraints in a planning or optimisation problem. [7] proposed a contact-implicit trajectory optimisation framework based on complementarity constraints, allowing the planner to reason about contact switches directly. Such approaches are attractive because they provide a principled way to predict the effects of pushing and to enforce physical constraints during planning.

Their main strength is precision: when object geometry, friction coefficients, and contact conditions are known, they can generate highly structured and predictable motions. Their main limitation is sensitivity to modelling error. In practice, uncertainty in friction, object mass distribution, or contact state can substantially degrade performance, which limits deployment in unstructured environments.

3.3.2 Sampling-based planning

Dogar and Srinivasa [8] formulated pushing as a motion-planning problem in the joint configuration space of the robot and object. Their work showed that reliable pushing requires explicit reasoning about the contact mode, including sliding, sticking, and rolling transitions. This line of work is important because it highlights that pushing is not just a local controller design problem but also a global planning problem under hybrid contact dynamics.

3.3.3 Predictive control

A more reactive but still structured family of methods relies on predictive models. Hogan and Rodriguez [9] formulated planar pushing as a contact-implicit model predictive control problem. By replanning online with a linearised contact model, their method can react to disturbances and changing environmental conditions while still enforcing friction-related constraints.

Learning-based predictive control follows a similar philosophy but replaces the analytic model with a learned forward model. Finn and Levine [10] illustrate this trend: the robot learns a predictive model of the effect of its actions from raw interaction data and then uses this model for planning. Compared with end-to-end policy learning, this approach retains a modular structure and is often easier to interpret, but its performance depends strongly on the quality of the learned dynamics model.

3.3.4 Feedback and reactive control

At the opposite end of the spectrum are reactive strategies that do not attempt to predict long-horizon object motion in detail. Lloyd and Lepora's tactile pushing work [11], alongside Lynch et al.'s earlier tactile-feedback pushing study [12], illustrates this idea well: rather than relying on a precise contact model, the controller uses tactile and proprioceptive feedback to maintain stable contact during pushing.

The main advantage of such feedback-based strategies is robustness to local uncertainties such as surface variations, object shape irregularities, and contact perturbations. Their main limitation is that they are primarily local: they react well to immediate contact events but generally do not optimise over long horizons.

More recent work continues along both lines: UNO Push [13] combines non-parametric contact estimation with MPC, while learned-policy approaches [14, 15, 16] apply learned policies to non-prehensile manipulation.

3.4 Force-feedback and compliant pushing control

Force information is especially important in pushing because the quality of the interaction depends directly on whether contact is maintained, lost, or driven into an unstable regime. However, many robots do not have dedicated wrist force/torque sensors, which has motivated both compliant control strategies and force-estimation methods.

3.4.1 Impedance and admittance control

Hogan [17] introduced impedance control as a general framework for contact-rich manipulation. Instead of prescribing a perfectly rigid position trajectory, the robot is assigned a desired mechanical behaviour, typically that of a mass, spring and damper system. In pushing, this allows the manipulator to absorb small positioning errors and environmental uncertainties while still maintaining effective contact.

Villani and De Schutter [18] surveyed the broader family of force control methods and clarified the distinction between direct force control and impedance-based interaction control. This distinction remains important in modern pushing systems, especially when contact stability matters as much as tracking accuracy.

3.4.2 Reactive force-feedback pushing

The most relevant prior work to this thesis is [19], who proposed a reactive force-feedback controller for goal-directed planar pushing. Their method operates directly in end-effector velocity space and uses the measured contact force direction as a correction signal for the desired pushing direction.

This controller is interesting because it is simple, computationally cheap, and does not require a full object model. At the same time, its original validation assumed a force/torque sensor on an omnidirectional mobile base rather than an arm-mounted spherical pusher. This assumption is precisely where the present thesis departs from prior work. The pusher is shown in Figure 3.2.

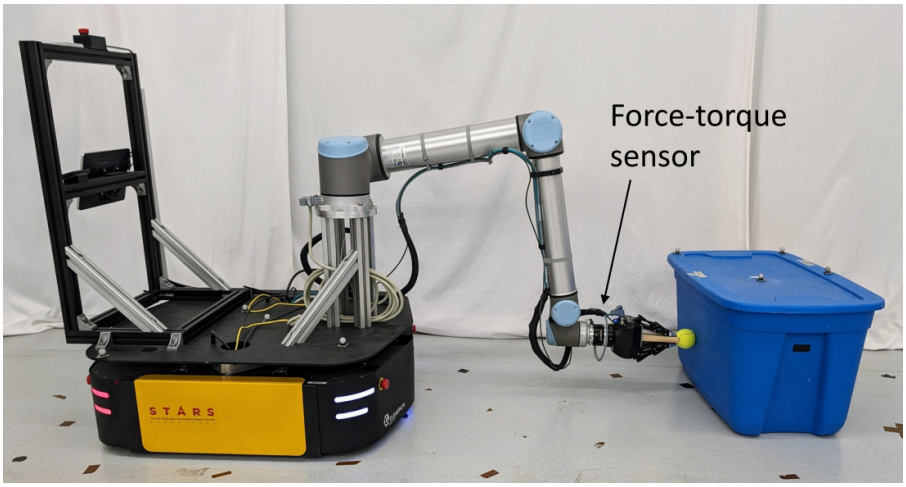


Figure 3.2: Non-prehensile pushing [19]

3.4.3 Force estimation without wrist sensors

A contact-force measurement can be available at the end-effector, as provided by a wrist force/torque sensor like [19] or by a tactile fingertip [11].

When direct force sensing is unavailable, contact forces may be estimated from joint torques through the robot dynamics [20], or inferred more indirectly from geometry and overlap [21]. The latter direction is particularly relevant when the end-effector geometry is simple and the contact surface is known approximately.

3.5 Perception for manipulation

Reliable pushing requires knowledge of the object pose. Classical pipelines use point-cloud segmentation and registration; a standard reference is the ICP algorithm of Besl and McKay [22], which remains foundational for 3-D alignment.

More recent approaches use learned object detection and pose estimation. LINEMOD [23] is a representative model-based method for texture-less object detection and pose estimation, while PoseCNN [24] illustrates learning-based RGB pose estimation. DenseFusion [25] further combines RGB and depth features for robust 6-DoF pose estimation in cluttered scenes. More broadly, Bohg *et al.* [26] emphasised that perception and manipulation should not be treated as independent modules: action can improve perception, and perception can guide action.

In pushing specifically, perception quality strongly influences controller performance because small pose errors lead directly to contact-location errors. At the same time, visual perception degrades during physical interaction due to occlusions, motion blur, and manipulator-induced clutter. This makes purely vision-based pipelines brittle during sustained contact.

3.6 Summary table

Table 3.1 summarises the main approaches reviewed in this chapter, together with their main strengths, limitations, and relevance to the present thesis.

Table 3.1: Summary of the main approaches for planar non-prehensile pushing.

Approach	Main idea	Strengths	Limitations
Mechanical models	Pushing mechanics and friction laws [4, 6]	Physical insight; foundation	Model-dependent
Model-based planning	Dynamics and contact constraints in optimisation / MPC [7, 9]	Structured plans; constraints	Sensitive to uncertainty
Sampling-based planning	Search over robot–object contact modes [8]	Good for long-horizon tasks	Computational cost
Reactive feedback	Local regulation from force / tactile feedback [11, 19]	Robust; simple; lightweight	Little anticipation
Compliant control and force estimation	Impedance/admittance with estimated contact forces [17, 18, 20, 21]	Relevant to contact-rich tasks	Tuning-sensitive
Perception	Pose estimation from geometry or vision [22, 23, 24, 25, 26]	Essential in real settings	Fragile under occlusion

4 System model

This chapter develops a general mathematical model of the robot, the pushed object, and the contact that couples them. The goal is to obtain a representation precise enough to state clearly what is being assumed, what limits the system is subject to, and how a contact force turns into motion on both sides of the interaction, before specialising to the quasi-static, single-point case that the rest of this thesis relies on.

4.1 Manipulator-object model

Understanding how the manipulator and the pushed object influence each other requires three ingredients: a dynamic model for each of the two bodies taken separately, a parametrisation of how contact forces are transmitted between them, and a combined model obtained by coupling the two through that parametrisation. This section develops each in turn.

4.1.1 Robot model

The Franka Panda is a serial manipulator with $n = 7$ revolute joints, its state fully specified by the pair $(q, \dot{q})$, $q \in \mathbb{R}^7$. Its dynamics follow the standard rigid-body form

$$M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + g(q) = \tau + \tau_{\text{ext}}, \tag{4.1}$$

where $M(q) \in \mathbb{R}^{7\times 7}$ is the symmetric positive-definite joint-space inertia matrix, $C(q,\dot{q})\dot{q}$ collects the Coriolis and centrifugal terms, $g(q)$ is the gravity vector, τ is the commanded joint torque, and τ_{ext} is the joint torque induced by an external load, here the reaction force at the contact between the end-effector and the object.

4.1.2 Object model

To obtain the object dynamic model, we can consider a system consisting of a single point contact between the end-effector and the object. Modelling the object requires stating assumptions in order to have a simplified system.

1. The object is a rigid body with known dynamical properties (mass m and planar moment of inertia I_{zz}, or equivalently the squared radius of gyration $c^2 = I_{zz}/m$).

2. The object rests on a flat, horizontal table and remains in contact with it throughout the task, so that the configuration is fully described by a planar pose $x_o = (x_o, y_o, \theta_o) \in SE(2)$ rather than the full six-degree-of-freedom rigid-body pose.

3. The robot interacts with the object through a single contact point, located on the object's boundary.

4. Only linear, in-plane forces are transmitted through this contact point: a point-contact-with-friction model, following the classical Coulomb contact model. No moment is transmitted about the contact normal, explained by the spherical pusher against a smooth object surface.

5. The object's motion is quasi-static.

6. Coulomb friction applies at both interfaces of the problem, each governed by its own known, uniform coefficient: μ_p between the pusher and the object, and μ_{table} between the object and the table.

Under Assumptions 1-2, the object's in-plane configuration is fully captured by the planar twist $V_o = (\dot{x}_o, \dot{y}_o, \dot{\theta}_o)^\top \in \mathbb{R}^3$, and its dynamics follow the planar Newton-Euler equation, the object-side analogue of Eq. (4.1),

$$M_o \dot{V}_o + C_o(V_o)\, V_o = F_o, \tag{4.2}$$

where $M_o = \text{diag}(m, m, I_{zz})$ is the constant planar mass matrix, $C_o(V_o)$ collects the terms arising from the rotation of the body-fixed frame relative to the world (the object-side analogue of the Coriolis term $C(q, \dot{q})\dot{q}$, here induced by the object's own rotation), and $F_o \in \mathbb{R}^3$ is the net planar wrench applied to the object. Under Assumptions 3–4, this wrench reduces to the single-contact form $F_o = G f_c$ already introduced in Eq. (4.4), giving the version used in Section 4.1.6. As noted there, Eq. (4.2) is never solved online as a genuine dynamic equation: under Assumption 5 (quasi-static motion), the acceleration term $\dot{V}_o$ is negligible compared with the contact and friction forces involved, and the equation collapses to the direct algebraic relation that the reactive controller is built around.

4.1.3 Contact model parametrization

In the general case of n_c contact points, as in a multi-fingered or tray-based grasp, the net object wrench is obtained through a grasp matrix $G \in \mathbb{R}^{6 \times d n_c}$ mapping the stacked contact forces F_c to the resultant wrench at the object's center of mass, $F_o = G F_c$, with each contact's contribution weighted by its lever arm relative to the object frame. Assumption 3 restricts this thesis to the degenerate case $n_c = 1$, and Assumption 4 restricts the transmissible force at that single contact to a planar linear force with $d = 2$ components, $f_c = (f_n, f_t)^\top$, decomposed into a normal and a tangential component in the local contact frame. The contact location

itself is parametrised, in the object's body frame, by a scalar boundary parameter s,

$$r_c(s) = \begin{pmatrix} r_x(s) \\ r_y(s) \end{pmatrix}, \tag{4.3}$$

with the shape-specific forms of $r_c(s)$ given in Sections 4.1.4 and 4.1.5. The grasp matrix reduces, in this single-contact case, to a 3×2 matrix mapping the local contact force directly to the planar object wrench,

$$F_o = G f_c, \qquad G = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ -r_y(s) & r_x(s) \end{pmatrix}, \tag{4.4}$$

which is exactly the wrench decomposition $\tau_o = r_x(s) f_t - r_y(s) f_n$.

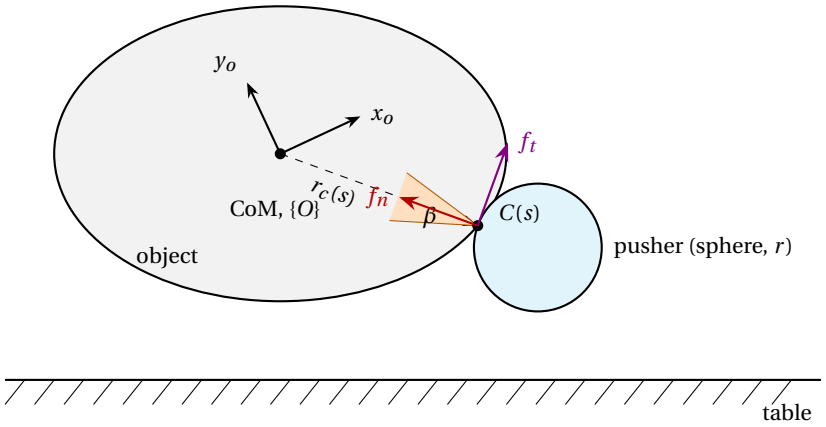


Figure 4.1: Contact geometry and force decomposition for the general system model. A convex object with body frame $\{O\}$ at its center of mass is pushed by a spherical end-effector at a single contact point $C(s)$, parametrised by the boundary variable s (Eq. (4.3)). The contact force decomposes into a normal component f_n and a tangential component f_t, constrained within a friction cone of half-angle $\beta = \arctan(\mu_p)$ (Eq. (4.20)). The vector $r_c(s)$ locates the contact relative to the center of mass and feeds directly into the grasp-matrix relation of Eq. (4.4); θ_o is the object's orientation relative to the world frame.

4.1.4 Cube: contact geometry and force/torque model

Under the quasi-static assumption (Assumption 5, Section 4.1.2), the object's motion at each instant is treated as a succession of force balances: inertial terms are negligible next to the contact and friction forces. This is the model that both this section and Section 4.1.5 instantiate for a specific shape: the contact wrench is decomposed into a normal and a tangential component at the single contact point of Assumption 3, and combined with the object's radius of gyration to characterise how that wrench turns into translation and rotation.

A cube offers a discrete set of four flat faces to push against. Unlike the cylinder (Section 4.1.5), where the contact point is fixed by geometry alone, a flat face leaves the contact location genuinely free within it: the pusher can be well-centered on the face or offset by some amount p_y, and it is precisely this offset that

turns out to be the dominant source of unwanted torque. In the implementation, the active face is chosen once, at planning time, and held fixed for the remainder of the push; only the object's own yaw θ_o is tracked afterwards, through the same low-pass filter used for the desired direction (Section 5.2), so that the frozen face axis stays consistent with the object's slowly rotating body frame rather than with its planning-time orientation alone.

Contact geometry. Expressed in the object's body frame, with the local x-axis aligned with the active face's outward normal, the contact location is

$$r_c = \begin{pmatrix} a \\ p_y \end{pmatrix}, \tag{4.5}$$

where a is the cube's half-side (fixed by the object's geometry) and $p_y \in [-a, a]$ is the lateral offset of the contact point from the face center. It is free in principle and, in practice, set by wherever the approach planner and the reactive controller happen to place the pusher relative to the object.

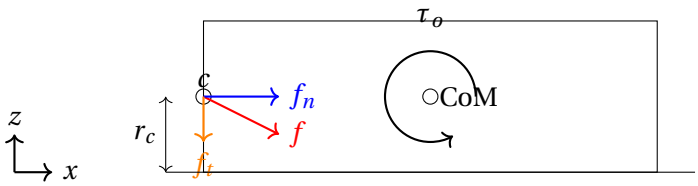


Figure 4.2: Side view of the pushing interaction for the cube. The contact point c, at height r_c above the table, receives the raw contact force f, decomposed into a normal component f_n and a tangential component f_t; the resulting torque about the center of mass is τ_o.

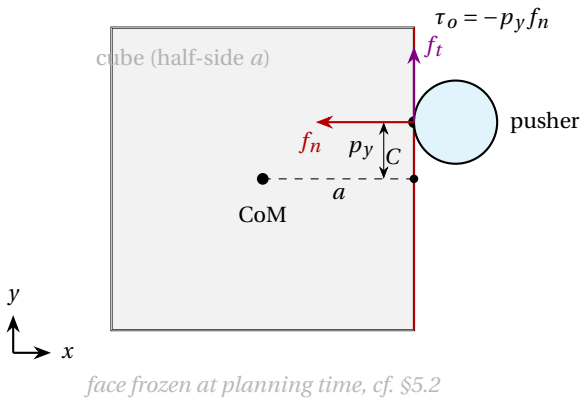


Figure 4.3: Top view of the pushing interaction for the cube. The active face (red) is chosen once, at planning time, and its contact point C can sit anywhere within it, offset from the face center by p_y (Eq. (4.5)). This lateral offset is what breaks the cylinder's exact zero-torque property (Section 4.1.5): here f_n no longer passes through the center of mass unless $p_y = 0$.

Force and torque decomposition. Substituting the contact location of Eq. (4.5) into the general torque relation $\tau_o = r_x f_t - r_y f_n$ of Eq. (4.4) gives, in general,

$$\tau_o = a f_t - p_y f_n. \tag{4.6}$$

For a face-aligned push, the pusher moves predominantly along the face normal, so the tangential term stays small relative to the off-center normal-force term whenever the contact is reasonably close to centered; the implementation retains only the dominant term,

$$\tau_o \approx -p_y f_n, \tag{4.7}$$

as explained by in Table 4.1. This is a modelling simplification, not an exact identity, unlike the cylinder, where $\tau_o = R f_t$ (Eq. (4.11)) holds exactly by construction because f_n always passes through the center. For the cube, Eq. (4.7) is only as good as the assumption that p_y stays small and that sliding along the face is limited; a large offset or a slipping contact would reintroduce the $a f_t$ term.

Radius of gyration. For a cube of mass m, half-side a (so a square footprint of side $2a$), height h, and uniform density $\rho = m/(4a^2 h)$, the planar moment of inertia about the vertical axis through the center of mass is

$$I_{zz} = \int_0^h \int_{-a}^{a} \int_{-a}^{a} \rho\,(x^2 + y^2)\,dx\,dy\,dz = \rho\,h \left[\frac{2a^3}{3} \cdot 2a + \frac{2a^3}{3} \cdot 2a \right] = \rho\,h \cdot \frac{8a^4}{3} = \frac{2}{3} m a^2, \tag{4.8}$$

$$c^2 = \frac{I_{zz}}{m} = \frac{2}{3} a^2. \tag{4.9}$$

As for the cylinder (Eq. (4.13)), this closed-form value does not depend on the object's height, and is computed at initialisation and logged for diagnostics, it does not feed any closed-loop term, the controller regulates the contact through the lateral centering and braking terms of Section 5.2 instead. A smaller a at constant mass gives a smaller c^2 and hence less rotational inertia to resist the torque of Eq. (4.7).

The full side-by-side comparison with the cylinder, including this result, is collected in Table 4.1 at the end of Section 4.1.5.

4.1.5 Cylinder: contact geometry and force/torque model

The cube's flat faces leave the contact location within a face genuinely free: the pusher can be well-centered or offset by some amount p_y, and that offset is precisely what Section 4.1.4 identifies as the source of unwanted torque. A cylinder of radius R removes this degree of freedom entirely. Because its boundary is equidistant from the center in every direction, the point where a spherical pusher touches the cylinder is fixed by geometry alone: it lies on the line joining the sphere center to the cylinder axis, projected onto the table plane. There is no equivalent, for the cylinder, of choosing to push slightly off-center on a face; every contact point is already exactly at radius R from the object's center.

Contact geometry. Expressed in the object's body frame, with the local x-axis aligned with the current contact normal $\hat{n}_{\text{face}}$, this fixes the contact location at

$$r_c = \begin{pmatrix} R \\ 0 \end{pmatrix}. \tag{4.10}$$

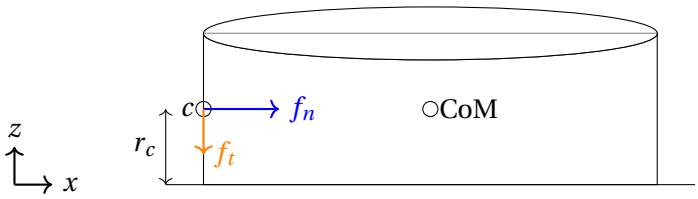


Figure 4.4: Side view of the pushing interaction for the cylinder, drawn on the same layout as Fig. 4.2 for direct comparison. Unlike the cube, no separate raw force vector f is shown decomposed away from f_n/f_t: because the contact point is fixed at $r_c = (R, 0)$ (Eq. (4.10)), f_n passes through the center of mass and contributes no torque; only f_t does, per $\tau_o = R f_t$ (Eq. (4.11)).

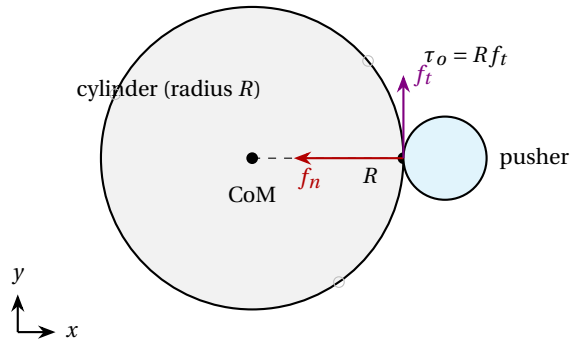


Figure 4.5: Top view of the pushing interaction for the cylinder. Because the boundary is equidistant from the center in every direction, the contact point C always lies at radius R from the center of mass (Eq. (4.10)), with no equivalent of the cube's lateral offset p_y. The faint markers indicate that, unlike the cube's frozen face axis, the active contact direction is re-evaluated at every control tick.

Force and torque decomposition. As for any convex primitive under the quasi-static model of Section 4.1.4, the contact wrench decomposes into a normal component f_n, directed along the inward radial direction, and a tangential component f_t, orthogonal to it in the table plane. Substituting the contact location of Eq. (4.10) into the general torque relation $\tau_o = r_x f_t - r_y f_n$ gives

$$\tau_o = R\, f_t. \tag{4.11}$$

This result is qualitatively different from the cube. Because the normal force always passes through the object's center by construction, f_n contributes no torque at all here, and the entire rotational effect of the push is carried by the tangential, frictional force component. For the cube, by contrast, the dominant source of unwanted torque is exactly the opposite: a normal force applied off-center on an otherwise flat face, $\tau_o = -p_y f_n$ (Section 4.1.4). The two shapes therefore fail differently under the same controller, which

is consistent with what was observed during the benchmark campaign: the cube tends to accumulate a slow rotational drift even under low friction, since it only takes a small, purely normal offset to start it turning, whereas a cylinder only starts rotating once the tangential component becomes significant, which is a narrower and more abrupt regime; closer to the near-instantaneous directional flick than to a gradual drift. This is offered here as a plausible mechanical explanation rather than a fully isolated cause, since the benchmark data also show other contributing factors.

Radius of gyration. The quasi-static pushing model requires the object's squared radius of gyration $c^2 = I_{zz}/m$, where I_{zz} is the moment of inertia about the vertical axis through the center of mass. For a solid cylinder of mass m, radius R, height h, and uniform density $\rho = m/(\pi R^2 h)$,

$$I_{zz} = \int_0^h \int_0^{2\pi} \int_0^R \rho\, r^2 \cdot r\, dr\, d\theta\, dz = \rho\, h \cdot 2\pi \cdot \frac{R^4}{4} = \frac{1}{2} m R^2, \tag{4.12}$$

$$c^2 = \frac{I_{zz}}{m} = \frac{R^2}{2}. \tag{4.13}$$

This is exact for a uniform cylinder and, notably, does not depend on the height h: the planar pushing model only ever involves rotation about the vertical axis, so the radius alone sets the object's rotational inertia in the plane, unlike I_{xx} and I_{yy}, which do depend on h and are only relevant to the three-dimensional rigid-body simulation itself rather than to the planar pushing model.

In the current controller, c^2 is computed for both shapes at initialisation and logged for diagnostic purposes. It is the physically correct quantity from Lynch and Mason's limit-surface analysis [5], and Eq. (4.13) is the closed-form result that the shape-generalisation code relies on, but it does not currently feed into an active, closed-loop correction term; the controller regulates the contact through the lateral centering and braking terms of Section 5.2 instead.

Summary. Table 4.1 collects the two derivations side by side.

	Cube (half-side a)	Cylinder (radius R)
Contact location r_c	(a, p_y), p_y free within the face	$(R, 0)$, fixed by geometry
Torque τ_o	$-p_y f_n$	$R f_t$
Dominant torque source	normal force, off-center	tangential (frictional) force
Radius of gyration c^2	$\frac{2}{3}a^2$	$\frac{1}{2}R^2$

Table 4.1: Contact geometry and force/torque model, cube versus cylinder.

4.1.6 Combined dynamic model

In order to couple the two subsystems, the contact force f_c is transformed into the corresponding joint torque through the manipulator's translational Jacobian at the contact point (the sphere center, since the

sphere's rigid attachment to `panda_hand` places the contact directly at a known offset from it),

$$\tau_{\text{ext}} = J_v(q)^\top f_{c,\text{world}}, \tag{4.14}$$

where $f_{c,\text{world}}$ is the contact force expressed in world coordinates. Substituting Eq. (4.14) into Eq. (4.1), and Eq. (4.4) into Eq. (4.2), gives the coupled system

$$M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + g(q) = \tau + J_v(q)^\top f_{c,\text{world}}, \tag{4.15}$$

$$M_o\,\dot{V}_o + C_o(V_o)\,V_o = G f_c. \tag{4.16}$$

Equations (4.15)-(4.16) are the general foundation of the system: they hold regardless of how f_c is generated. They are not solved online as such; under the quasi-static assumption (Assumption 5), Eq. (4.16) collapses to the direct kinematic relation already noted in Section 4.1.2, and it is this simplified relation, not the full dynamic equation, that the reactive controller is built around. Equation (4.15), by contrast, is used as stated: it is exactly the relation that maps the commanded velocity to a joint torque through the impedance law of Eq. (5.4).

4.2 Manipulator-object constraints

The combined model of Section 4.1 is only physically meaningful within the limits of what the robot can actually do and what the contact can actually sustain. These two families of constraints are treated separately below.

4.2.1 Robot constraints

The Franka Panda is subject to three standard types of constraint.

Actuator saturation. Each joint torque is bounded by the motor's physical limits,

$$-\tau_{\max,i} \le \tau_i \le \tau_{\max,i}, \qquad i = 1,\dots,7, \tag{4.17}$$

where $\tau_{\max,i}$ is the torque limit of joint i (Table 5.1).

Operating range. Each joint angle is bounded by the manipulator's mechanical limits,

$$q_{\min,i} \le q_i \le q_{\max,i}, \qquad i = 1,\dots,7, \tag{4.18}$$

where $q_{\min,i}$ and $q_{\max,i}$ are the manufacturer-specified joint limits.

Speed limitations. Each joint velocity is similarly bounded,

$$\dot{q}_{\min,i} \le \dot{q}_i \le \dot{q}_{\max,i}, \qquad i = 1,\dots,7. \tag{4.19}$$

where $\dot{q}_{\min,i}$ and $\dot{q}_{\max,i}$ are the joint velocity limits.

4.2.2 Contact and friction constraints

Two friction interfaces are relevant here, and they play structurally different roles, unlike a grasp-based system where a single non-sliding condition must hold at every contact.

Pusher-object contact. The tangential force cannot exceed the local friction cone,

$$|f_t| \le \mu_p f_n, \qquad f_n \ge 0, \tag{4.20}$$

where μ_p is the friction coefficient of the object. Non-penetration between the sphere and the object is enforced by the simulator's compliant Hunt-Crossley contact model rather than by an exact rigid-body complementarity condition ($\delta \ge 0$, $f_n \ge 0$, $\delta \cdot f_n = 0$ in the ideal rigid limit).

Object-table interface. Unlike the pusher-object contact, sliding at this interface is not something to be prevented: it is the mechanism by which pushing works at all. What matters instead is that the combined friction coefficient,

$$\mu_c = \frac{2\,\mu'_p\,\mu_{\text{table}}}{\mu'_p + \mu_{\text{table}}}, \tag{4.21}$$

(the harmonic mean Drake uses to combine two surfaces' friction coefficients) matches the object's observed steady-state sliding behaviour.

Unlike a grasp-based system model, where the contact forces are decision variables of an online optimisation and must therefore be explicitly parametrised as a linear combination of polyhedral friction-cone edges to keep that optimisation convex, the controller of this thesis does not solve for f_c at all: it is measured or estimated (Section 5.2.3), and the friction cone of Eq. (4.20) is used as an analysis and validation constraint.

5 Controller design

5.1 Approach and repositioning planner

Before the controller can regulate contact, the end-effector first has to reach the object without colliding with it. This is handled by a fixed-structure planner: given the object's position and the goal, it produces a short sequence of Cartesian waypoints that the existing position controller tracks open-loop, and it hands the controller the push direction, the planned push distance, and the geometric contact normal. With these informations the controller can start pushing the object.

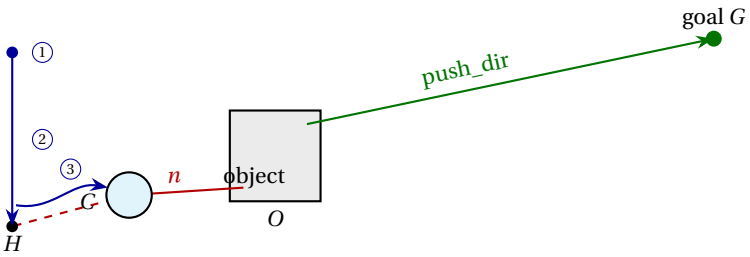
Approach geometry. Let $O \in \mathbb{R}^2$ be the planar object position and $G \in \mathbb{R}^2$ the planar goal position. The unit push direction is

$$\widehat{dir} = \frac{G - O}{\|G - O\|}, \tag{5.1}$$

where $\widehat{dir}$ is the normalised vector from the object to the goal and $\|\cdot\|$ is the Euclidean norm. The approach side is the opposite one: the unit contact normal n points from the object centre toward the side approached by the pusher, so that pushing along $-n$ drives the object toward G. For the cylinder, $n = -\widehat{dir}$. For the cube, n is the nearest face normal, accounting for the current object orientation. The contact point C is the point where the sphere touches the object, while the hover point H is a collision-free point farther along n.

Waypoint sequence. The approaching state tracks exactly three waypoints, listed in Figure 5.1: a safety-height point above H, a descent to pushing height still at H, and a final lateral move to C. This vertical then lateral structure keeps the sphere from clipping the object's top edge while descending, at the cost of assuming the space directly above H is itself free, which holds in the tabletop scenes considered here but would need revisiting for cluttered scenes.

In case the contact is lost during the push phase, the same construction is reused for the reposition state, extended with two additional waypoints at the front: the sphere's current position (held at safety height) and a retreat point along n, offset far enough from the object to clear it before re-approaching through the same hover-then-contact sequence as above. Both n and the push direction are recomputed from the object's



1 (H_{xy}, z=APPROACH_Z)
2 (H_{xy}, z=Z_PUSH)
3 (C_{xy}, z=Z_PUSH)

APPROACH state: 1 → 2 → 3; on reaching point 3 (C), the FSM switches to PUSH and follows push_dir (Section 5.2).

Figure 5.1: The three-waypoint approach trajectory. The end-effector first rises above the object at the hover point H (waypoint 1, safety height), descends at that same (x, y) to pushing height (waypoint 2), then moves in laterally to the contact point C on the object's boundary (waypoint 3). Reaching waypoint 3 hands control to the reactive push controller of Section 5.2, which then drives the object along push_dir toward G.

position at the moment of repositioning rather than reused from the original plan, which is also what the dead-reckoning distance of Eq. (5.17) needs to be refreshed to stay valid after a reposition (Section 5.3).

Waypoint tracking. Each waypoint w_k is tracked by a dedicated position controller, active throughout the approach phase, which implements the Cartesian PD law:

$$F_{\text{pos}} = K_{p,\text{pos}}\left(w_k - p_s\right) - K_{d,\text{pos}}\dot{p}_s + F_{\text{floor}}, \tag{5.2}$$

where F_{pos} is the Cartesian force command, $K_{p,\text{pos}}$ and $K_{d,\text{pos}}$ are respectively the position and damping gains, w_k is the current Cartesian waypoint, p_s and $\dot{p}_s$ are the sphere position and velocity, and F_{floor} is the vertical floor-compensation force. The force is mapped to joint torque through $\tau = J_v(q)^\top F_{\text{pos}} - g(q)$, where τ is the commanded joint-torque vector, $J_v(q)$ is the translational Jacobian at joint configuration q, and $g(q)$ is the gravity-torque vector.

5.2 Push controller

Figure 5.2 summarises the information flow through the complete system. The object pose $x_o(t_0)$ is sampled once by the Camera block according to Eq. (5.18), producing the frozen noisy estimate $\hat{x}_o$. From this estimate, the Planner computes the filtered desired push direction $\hat{d}$ and the waypoint sequence w_k described in Section 5.1. The Controller converts these references into the Cartesian velocity command v_{cmd} sent to the Robot. During the interaction, the estimated contact force $\hat{f}_c$ is fed continuously to the Controller, independently of the camera measurement, while the joint position q and joint velocity $\dot{q}$ close the robot's inner feedback loop.

The FSM of Section 5.1 is served by two distinct control laws, switched according to the current state

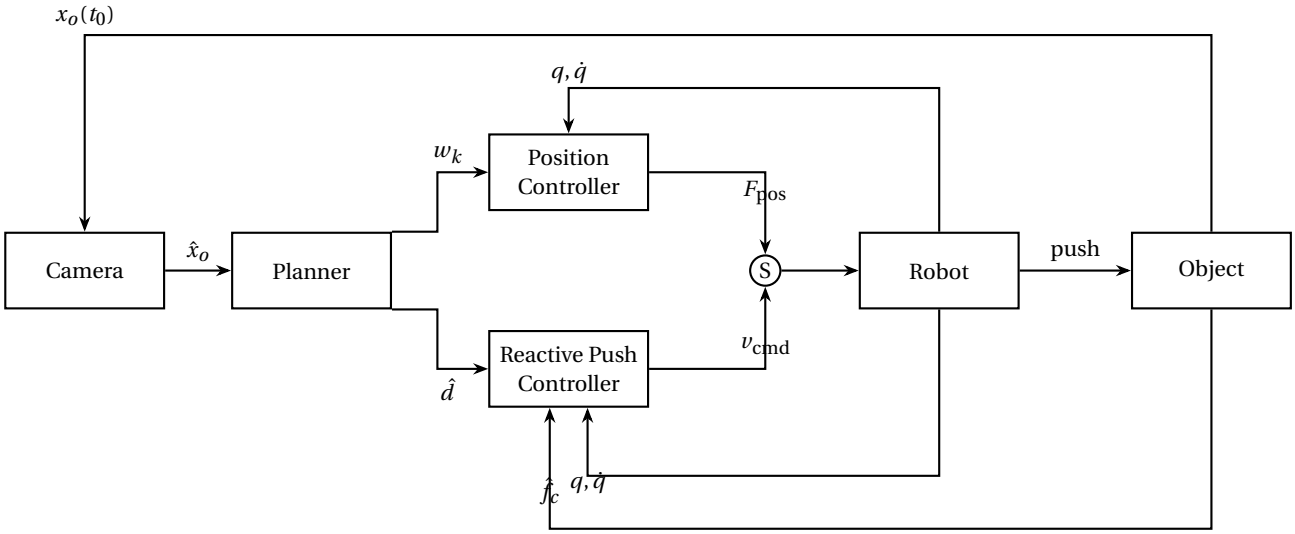


Figure 5.2: Control architecture (Section 5.2), showing the two control laws switched by the FSM. *S*: the FSM-driven selector, APPROACH route $F_{\text{pos}} = K_{p,\text{pos}}(w_k - p_s) - K_{d,\text{pos}}\dot{p}_s + F_{\text{floor}}$ (a Cartesian PD tracking the current waypoint w_k) to the Robot; PUSH routes v_{cmd} from the Reactive Push Controller instead, itself tracked through the velocity-impedance law of Figure 5.3. Both branches consume the joint state $q, \dot{q}$ (each needs $\dot{p}_s = J_v(q)\dot{q}$ for its own damping/feedback term); only the Reactive Push Controller consumes the force estimate $\hat{f}_c$, since the Position Controller never operates in sustained contact. $x_o(t_0)$, $\hat{x}_o$: as in the single-shot camera model of Eq. (5.18).

(Figure 5.2). During the approach phase, before contact is established, a Cartesian PD law tracks the current waypoint w_k directly, following the PD law of (Eq. (5.2)). This position controller plays the same role as the stabilizing controller of a grasp-based system operating without an external load: it has no notion of the object or of contact, and simply drives the sphere to a fixed reference point. It is deliberately kept this simple, since nothing about the push task is relevant before contact is made.

Once the last waypoint is reached, the FSM switches to the pushing phase and control passes to the pushing controller developed in the remainder of this section. This controller does not track a fixed point as it commands a cartesian velocity $v_{\text{cmd}} \in \mathbb{R}^3$ for the end-effector, continuously re-derived from the (possibly perceived) object position and the measured contact force, and tracked through a velocity-impedance law (Eq. (5.4)).

$$\dot{p}_s = J_v(q)\,\dot{q}, \tag{5.3}$$

where $\dot{q}$ is the joint-velocity vector and $\dot{p}_s$ is the corresponding Cartesian velocity of the sphere. This velocity feeds the velocity-impedance law

$$F_{\text{trans}} = K_v\left(v_{\text{cmd}} - \dot{p}_s\right) + F_{\text{floor}}, \tag{5.4}$$

where F_{trans} is the virtual Cartesian force, K_v is the velocity-impedance gain, and v_{cmd} is the desired Cartesian sphere velocity. The resulting force is mapped to joint torque through $\tau = J_v(q)^\top F_{\text{trans}} - g(q)$ and saturated at the Franka Panda torque limits. Commanding torque through $J_v^\top$ rather than commanding joint velocity through J_v^{-1} keeps the end-effector compliant against an object whose exact position is not known in real time.

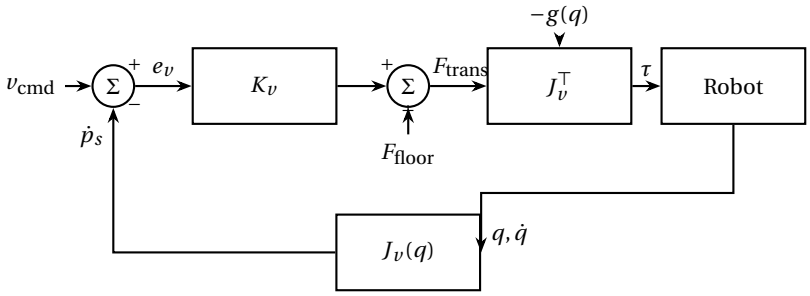


Figure 5.3: Velocity-impedance inner loop used by the Controller and Robot blocks of Figure 5.2.

Figure 5.3 details how the Cartesian command is converted into joint torques. The desired velocity v_{cmd} is compared with the measured sphere velocity $\dot{p}_s = J_v(q)\dot{q}$, where $J_v(q)$ is the translational Jacobian evaluated at the joint configuration q and $\dot{q}$ is the joint-velocity vector. Their difference is the velocity error e_v, which the impedance gain K_v converts into a virtual corrective force. Adding the floor-compensation term F_{floor} gives the total translational wrench F_{trans}, which is mapped to the commanded joint torque by $J_v^\top$; gravity compensation $g(q)$ is then subtracted, yielding $\tau = J_v^\top F_{\text{trans}} - g(q)$. The loop therefore uses the forward Jacobian to obtain Cartesian velocity and its transpose to map force into torque, without requiring an inverse or pseudo-inverse of J_v.

5.2.1 Direction filtering

At every tick the desired direction is recomputed from the (possibly perceived) object position, $\hat{d}_{\text{raw}} = (g - \hat{p})/\|g - \hat{p}\|$. Because $\hat{d}$ can reverse by more than 140° within about a second if the object slightly overshoots the goal, a behaviour observed empirically and referred to throughout this chapter as the *flick*, it is not used directly but low-pass filtered first,

$$\hat{d}_{\text{filt}} \leftarrow \frac{D_{\hat{d}}\,\hat{d}_{\text{raw}} + (1 - D_{\hat{d}})\,\hat{d}_{\text{filt}}}{\|D_{\hat{d}}\,\hat{d}_{\text{raw}} + (1 - D_{\hat{d}})\,\hat{d}_{\text{filt}}\|}, \tag{5.5}$$

where $\hat{d}_{\text{raw}}$ is the instantaneous unit direction from the perceived object position $\hat{p}$ to the goal g, $\hat{d}_{\text{filt}}$ is its filtered and renormalised value, and $D_{\hat{d}}$ is the direction-filter coefficient. The assignment arrow denotes an update at each control tick. The filtered direction defines $\theta_d = \text{atan2}(\hat{d}_y, \hat{d}_x)$, the desired heading, and $\hat{n}_d = [-\hat{d}_y, \hat{d}_x]^\top$, the unit direction perpendicular to the push.

When contact is detected, the direction is corrected by the force-feedback law of Heins et al. [19],

$$\theta_p = \theta_d + (K_F + 1)\,\delta_f + K_C\,\delta_c, \tag{5.6}$$

where θ_p is the commanded pushing heading, θ_d is the goal-directed heading, K_F is the force-direction gain, K_C is the lateral-offset gain, δ_f is the wrapped angular error between the force and desired directions, and δ_c is the signed lateral contact offset. More precisely, $\theta_f = \text{atan2}(f_{k,y}, f_{k,x})$ is the direction of the filtered force f_k, $\delta_f = \text{wrap}(\theta_f - \theta_d)$, and $\delta_c = \hat{n}_d^\top (p_s^{xy} - \hat{p})$, where p_s^{xy} is the planar sphere position. When contact

is not detected, $\theta_p = \theta_d$.

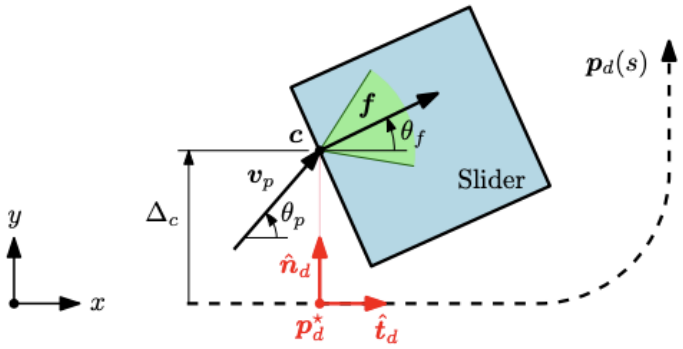


Figure 5.4: Geometry underlying the force-corrected pushing direction of Eq. (5.6), adapted from Heins et al. [19]. The desired direction is expressed through the local frame $(\hat{t}_d, \hat{n}_d)$ attached to the current target point $p_d^\star$ on the planned path $p_d(s)$; θ_p is the angle of the commanded pushing velocity v_p relative to $\hat{t}_d$, θ_f the angle of the measured contact force f relative to the same reference, and Δ_c the lateral offset of the contact point c from the path, the same quantity as δ_c in Eq. (5.6), expressed here in the path-relative frame rather than the world frame. Because θ_d fixes the orientation of $(\hat{t}_d, \hat{n}_d)$ in the world frame, the two formulations are equivalent; the world-frame form is what is implemented in Eq. (5.6).

5.2.2 Shape-dependent contact normal

The active contact normal $\hat{n}_{\text{face}}$ is handled differently for the two shapes.

Cylinder. The contact normal is re-derived every tick directly from the filtered direction, $\hat{n}_{\text{face}} = -\hat{d}_{\text{filt}}$ – safe because $\hat{d}_{\text{filt}}$ is already stabilised by Eq. (5.5).

Cube. The active face axis is instead locked at planning time, when the approach phase begins, and is never reselected from $\hat{d}$ during the push. Recomputing it at every tick, as for the cylinder, was tested and subsequently abandoned: if the object overshoots the goal and $\hat{d}$ reverses, the controller may switch to the opposite face even though the object has not actually rotated. This spurious face switch caused jumps of several centimetres near the end of multiple pushes. Instead, only the object's own yaw θ_o, measured relative to its value at the planning instant, is allowed to rotate the frozen axis, and θ_o itself is filtered first,

$$\hat{n}_{\text{face}} = R(\theta_{o,\text{filt}} - \theta_{o,\text{ref}})\, \hat{n}^0_{\text{face}}, \qquad \theta_{o,\text{filt}} \leftarrow \text{lowpass}(\theta_o, \Theta_o), \tag{5.7}$$

where $\hat{n}_{\text{face}}$ is the current cube-face normal, $R(\cdot)$ is the planar rotation matrix, $\theta_{o,\text{filt}}$ is the filtered object yaw, $\theta_{o,\text{ref}}$ is the yaw recorded when the push was planned, and $\hat{n}^0_{\text{face}}$ is the face normal selected at that instant. The measured yaw is θ_o, lowpass$(\cdot)$ denotes the yaw-filter update, and Θ_o is its filter coefficient.

5.2.3 Contact estimation and confirmation

Contact force is estimated from two sources. When Drake's own contact solver reports contact between the sphere and the object, the corresponding reaction force is used directly. Otherwise, a geometric fallback

estimates the force from the penetration δ between the sphere and the active face along $\hat{n}_{\text{face}}$,

$$f_{\text{raw}} = K_c\,\delta\,(-\hat{n}_{\text{face}}), \tag{5.8}$$

where f_{raw} is the unfiltered planar force estimate, K_c is the geometric contact stiffness, δ is the penetration depth, and $-\hat{n}_{\text{face}}$ is the estimated reaction-force direction. The fallback is gated behind an independent geometric proximity test. Either force source is then low-pass filtered,

$$f_k = \beta_f\, f_{\text{raw}} + (1-\beta_f)\, f_{k-1}. \tag{5.9}$$

Here, f_k is the filtered force at control tick k, f_{k-1} is the previous filtered force, and β_f is the force-filter coefficient.

Contact is confirmed when any of three independent signals is positive: Drake reports it directly, the geometric proximity test is satisfied, or the filtered force magnitude exceeds $F_{\min}$, the minimum force-detection threshold.

5.2.4 Lateral centering

A lateral centering velocity continuously drives the end-effector center to align with the object center in the direction perpendicular to the push,

$$v_{\text{lat}} = K_{\text{lat}}\,\varepsilon_{\text{lat}}\,\hat{n}_d, \qquad \varepsilon_{\text{lat}} = \hat{n}_d^{\top}(\hat{p} - p_s^{xy}). \tag{5.10}$$

where v_{lat} is the lateral correction velocity, K_{lat} is the lateral-centering gain, ε_{lat} is the signed lateral position error, $\hat{n}_d$ is the unit direction perpendicular to the push, $\hat{p}$ is the perceived object position, and p_s^{xy} is the planar sphere position.

5.2.5 Braking profile and velocity decomposition

The forward push speed decelerates as the object approaches the goal:

$$\text{frac} = \text{clip}\left(\frac{d - d_{\text{done}}}{d_{\text{brake}} - d_{\text{done}}},\ 0,\ 1\right), \tag{5.11}$$

$$v_{\text{profile}} = \max\left(V_{\text{push}}\sqrt{\text{frac}},\ V_{\text{creep}}\right), \tag{5.12}$$

where $d = \|\hat{p} - g\|$ is the perceived distance to the goal, d_{done} is the completion threshold, d_{brake} is the distance at which braking begins, frac is the normalised braking fraction bounded to $[0,1]$, V_{push} is the nominal push speed, V_{creep} is the minimum speed retained near the goal, and v_{profile} is the resulting forward-speed command. The operator clip bounds its first argument between the stated lower and upper limits. The commanded horizontal velocity then combines this profile with the force-corrected direction and the lat-

eral centering term,

$$v_{xy} = v_{\text{profile}} \begin{pmatrix} \cos\theta_p \\ \sin\theta_p \end{pmatrix} + K_{\text{lat}}\, \varepsilon_{\text{lat}}\, \hat{n}_d. \tag{5.13}$$

where v_{xy} is the commanded planar velocity and $(\cos\theta_p, \sin\theta_p)^\top$ is the unit vector associated with the commanded heading θ_p. A second correction damps only the object's *excess* velocity along the push direction, relative to what the profile currently allows, and leaves its legitimate velocity untouched,

$$v_{xy} \leftarrow v_{xy} - V_{\text{damp}} \max(0,\ v_{\text{obj}}^{\parallel} - v_{\text{profile}}) \begin{pmatrix} \cos\theta_p \\ \sin\theta_p \end{pmatrix}, \tag{5.14}$$

where V_{damp} is the excess-velocity damping gain and $v_{\text{obj}}^{\parallel}$ is the object's filtered velocity projected onto the push direction. The max term is nonzero only when the object moves faster than the active speed profile. Finally, an admittance saturation limits the commanded velocity when the measured force exceeds $F_{\max}$,

$$v_{xy} \leftarrow v_{xy} \frac{F_{\max}}{\|f_k\|} \quad \text{if } \|f_k\| > F_{\max}, \tag{5.15}$$

where $F_{\max}$ is the force threshold for velocity scaling and $\|f_k\|$ is the magnitude of the filtered contact force. The ratio preserves the velocity direction while reducing its magnitude. A force-independent hard cap is subsequently applied at $\gamma_v V_{\text{push}}$, where γ_v is the velocity-cap multiplier.

5.2.6 Vertical regulation

The vertical component keeps the sphere at the pushing height z_{push},

$$v_z = \text{clip}\big(K_{z,p}(z_{\text{push}} - p_{s,z}) - K_{z,d}\,\dot{p}_{s,z},\ -V_{z,\max},\ V_{z,\max}\big), \tag{5.16}$$

where v_z is the commanded vertical velocity, $K_{z,p}$ and $K_{z,d}$ are the vertical position and damping gains, z_{push} is the desired pushing height, $p_{s,z}$ and $\dot{p}_{s,z}$ are the sphere's vertical position and velocity, and $V_{z,\max}$ is the maximum vertical speed magnitude.

5.2.7 Termination and loss of contact

The pushing state ends through one of two criteria. The first one compares the perceived distance to the goal against the threshold d_{done}, so that $d < d_{\text{done}}$. The second is a dead-reckoning criterion based on the sphere's own displacement since the start of the push, along the direction planned at that time,

$$\text{sphere\_progress} = (p_s^{xy} - p_{s,\text{start}}^{xy})^\top \hat{d}_{\text{plan}} \ \geq\ \|g - \hat{p}_{\text{plan}}\|, \tag{5.17}$$

where sphere_progress is the sphere displacement projected along the planned push direction, p_s^{xy} is the current planar sphere position, $p_{s,\text{start}}^{xy}$ is its position at push onset, $\hat{d}_{\text{plan}}$ is the unit direction stored by the planner, g is the goal position, and $\hat{p}_{\text{plan}}$ is the perceived object position used to create the plan. The right-hand side is therefore the planned push distance.

If none of Drake, the geometric test, or the filtered force confirm contact for more than T_{nc}, the no-contact timeout, the controller re-plans an approach. If the object is already within $\gamma_{\text{done}} d_{\text{done}}$ of the goal, γ_{done} being the tolerant-completion multiplier, the push is accepted as complete to avoid irrelevant repositioning.

5.3 Camera localisation and dead reckoning

The controller of Section 5.2 does not need a continuously updated object position to maintain contact: the force channel already provides that information, on the physical channel, independently of vision. This suggests a deliberately minimal perception scheme, closer to what a single fixed camera can realistically provide once the manipulator starts occluding its own view of the object: one noisy localisation, captured before the push begins, rather than a continuous tracking pipeline running throughout the interaction.

Before settling on an explicit camera model, a cheaper alternative was considered: inferring the object's position purely from the end-effector's own kinematics, since the sphere's position is always known exactly and its offset from the object's surface is known by construction once contact is established. This proxy turns out to be structurally blind to the lateral centering term of Eq. (5.10). If the sphere itself has drifted laterally by some amount, the proxy inherits exactly that same drift, so the lateral error ε_{lat} computed from it cancels by construction regardless of how misaligned the true contact actually is. A kinematic proxy can track where the pusher is, but not whether the pusher is where it should be relative to the object, which is precisely what the lateral centering term needs to know.

Camera model. The object's position is sampled at the start of the push, by a synthetic camera model that reproduces realistic noise on that single measurement,

$$\hat{p} = p_{\text{true}}(t_0) + \eta, \qquad \eta \sim \mathcal{N}(0,\, \sigma^2 I_2), \tag{5.18}$$

where $\hat{p}$ is the measured planar object position, $p_{\text{true}}(t_0)$ is the true object position at capture time t_0, η is the additive measurement-noise vector, $\mathcal{N}(0, \sigma^2 I_2)$ is a zero-mean isotropic Gaussian distribution, σ is the position-noise standard deviation, and I_2 is the two-dimensional identity matrix.

Termination criterion. The primary termination criterion of Section 5.2, $\|\hat{p} - g\| < d_{\text{done}}$, compares the perceived object position $\hat{p}$ with the goal g, using d_{done} as the completion-distance threshold. Once $\hat{p}$ is frozen, this distance no longer evolves. This motivates the dead-reckoning criterion of Eq. (5.17), which uses the sphere's kinematic progress and the push distance planned from the initial camera capture.

Limitation. The dead-reckoning distance in Eq. (5.17) is planned once, at the moment the push begins. If contact is lost and the controller re-plans through the repositioning, the plan used for dead reckoning must be refreshed consistently with the new approach, or the termination criterion would silently keep using a distance computed from the object's much earlier position. This is handled at the implementation level by refreshing the planned direction and distance in the repositioning routine as well as in the initial approach, so that the criterion of Eq. (5.17) remains valid after a reposition and not only for a push that never loses contact.

5.4 Controller parameter values

Table 5.1 centralises all numerical values used by the planner, controller, contact estimator, termination logic, and camera model. The preceding equations therefore define only the structure of the control laws and refer to the symbols listed here.

These values fall into three groups according to how they were obtained. A first group is imposed by the hardware and the scene: the joint torque limits $\tau_{\max}$ come from the manipulator model, and the pushing and approach heights follow from the table geometry. A second group is fixed by an argument given elsewhere in this work: the sphere radius r was chosen to be smaller or equal to the cube's side and the cylinder radius, the success threshold d_{done} sits above the waypoint tolerance of the Cartesian position controller, the contact stiffness K_c sets the scale of the geometric estimate of Eq. (5.8), and the three filter coefficients β_f, $D_{\hat{d}}$ and Θ_o are ordered by the timescale of the disturbance each one addresses (Sections 5.2.1 and 5.2.2): milliseconds for the contact force, about a second for the desired direction, and several seconds for the object's own rotation.

The remaining group, the feedback gains, was tuned by hand. Each gain was adjusted on a single nominal configuration until the push converged without oscillation, and then held fixed for the simulations. No parameter was re-tuned per shape, mass, friction coefficient or initial position, so the performance differences reported there reflect the behaviour of one fixed setting.

Symbol	Value	Unit	Description
r	0.05	m	Sphere radius
z_{push}	0.10	m	Pushing height (sphere center)
z_{approach}	0.50	m	Approach clearance height
V_{push}	0.06	m/s	Nominal push speed
V_{creep}	0.008	m/s	Minimum speed near the goal
d_{brake}	0.35	m	Distance at which braking begins
K_c	700	N/m	Geometric contact stiffness
F_{min}	0.05	N	Contact detection threshold
F_{max}	30	N	Admittance saturation threshold
β_f	0.30	–	Force-filter coefficient
$D_{\hat{d}}$	0.08	–	Desired-direction filter coefficient
Θ_o	0.05	–	Cube-yaw filter coefficient
K_F	0.15	–	Force-direction gain
K_C	0.10	–	Lateral-offset gain
K_{lat}	1.5	s^{-1}	Lateral-centering gain
V_{damp}	0.8	–	Excess-velocity damping gain
γ_v	3	–	Hard velocity-cap multiplier
$K_{z,p}$	0.3	s^{-1}	Vertical position gain
$K_{z,d}$	0.04	–	Vertical damping gain
$V_{z,\text{max}}$	0.01	m/s	Maximum vertical speed
$K_{p,\text{pos}}$	400	N/m	Cartesian position gain
$K_{d,\text{pos}}$	80	N s/m	Cartesian damping gain
K_v	600	N s/m	Velocity-impedance gain
d_{done}	25	mm	Success threshold
γ_{done}	2	–	Tolerant-completion multiplier
T_{nc}	5.0	s	No-contact timeout
σ	3	mm	Camera-position noise standard deviation
τ_{max}	$(87, 87, 87, 87, 12, 12, 12)$	N m	Joint torque limits

Table 5.1: Numerical parameter values used by the controller and perception model.

6 Results

This chapter evaluates the controller and camera model of Chapter 5 in five steps: a cross-check of the simulation's contact physics against the analytic Coulomb prediction (Section 6.2); a systematic benchmark across object shape, mass, friction, and initial position (Section 6.3); an analysis of the two dominant failure modes uncovered during that campaign and of the fix each motivated (Section 6.4); an ablation isolating the contribution of the two corrective terms (Section 6.5); and an evaluation of robustness to degraded perception (Section 6.6).

6.1 Evaluation protocol

Both object shapes (cube, cylinder) are evaluated across four masses (0.5, 1.0, 1.5, 2.0 kg), three object-side friction coefficients ($\mu \in \{0.3, 0.5, 0.7\}$, against a fixed table friction of 0.5), and the two initial position/goal pairs A and B of Table 6.1, for a total of 48 runs. Positions C and D of the same table are used only for the ablation of Section 6.5.

Table 6.1: Initial-position / goal pairs used in the benchmark and ablation studies. Positions A and B define the benchmark grid; positions C and D are used for the controller-component ablation.

Position	Object (x, y) [m]	Goal (x, y) [m]	Angle to nearest cube face axis
A	(0.35, 0.10)	(0.55, 0.20)	$\sim 27^\circ$
B	(0.30, 0.10)	(0.55, 0.05)	$\sim 11^\circ$
C	(0.35, 0.05)	(0.55, 0.00)	$\sim 14^\circ$
D	(0.35, 0.05)	(0.55, 0.20)	$\sim 37^\circ$

Success criterion. A run is successful if the object's final distance to the goal is below $d_{done} = 25$ mm. This value was chosen to stay above the 20 mm waypoint tolerance that already bounds the Cartesian position controller's own precision elsewhere in the FSM, and it remains small relative to the tested objects (about 25% of the cylinder's radius, 33% of the cube's half-side).

Metrics. For every run, the following are recorded: success (binary), final distance to goal, completion time, the fraction of PUSH time during which contact was confirmed directly by Drake, and the mean/maximum lateral deviation of the object's trajectory from the theoretical straight line joining its start position to the goal.

6.2 Physical validation

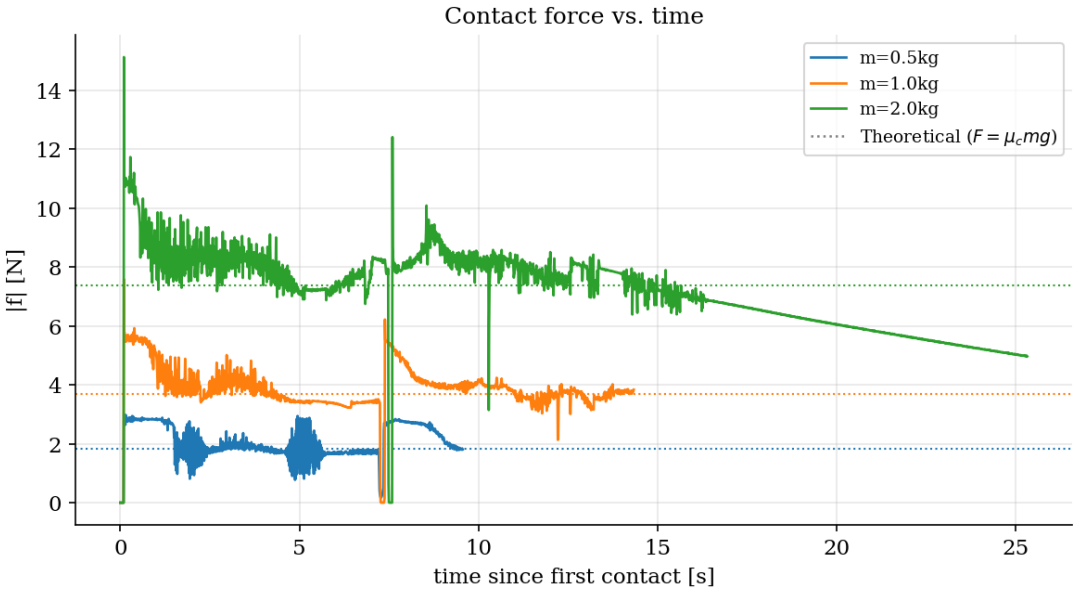
Before trusting any controller-level result, the simulation's contact physics were cross-checked against the analytic Coulomb friction prediction. For two surfaces in contact, Drake combines the two friction coefficients via the harmonic mean of Eq. (4.21) so that the theoretical steady-state sliding friction force is $F_{theory} = \mu_c\, m\, g$. For a table friction of 0.5 and an object friction of 0.3, $\mu_c = 0.375$; for a 2 kg object this gives $F_{theory} = 7.36$ N, against 6-9 N observed directly from Drake's `ContactResults` during steady sliding, which is a useful sanity check that the physics engine's contact model behaves as expected before ascribing any subsequent deviation to the controller itself rather than to the simulation. This cross-check was also what revealed that the default simulation time step over-estimated contact forces by a factor of 2 or 3, motivating the finer 10^{-4} s step used throughout the rest of this chapter.

6.2.1 Force and speed validation

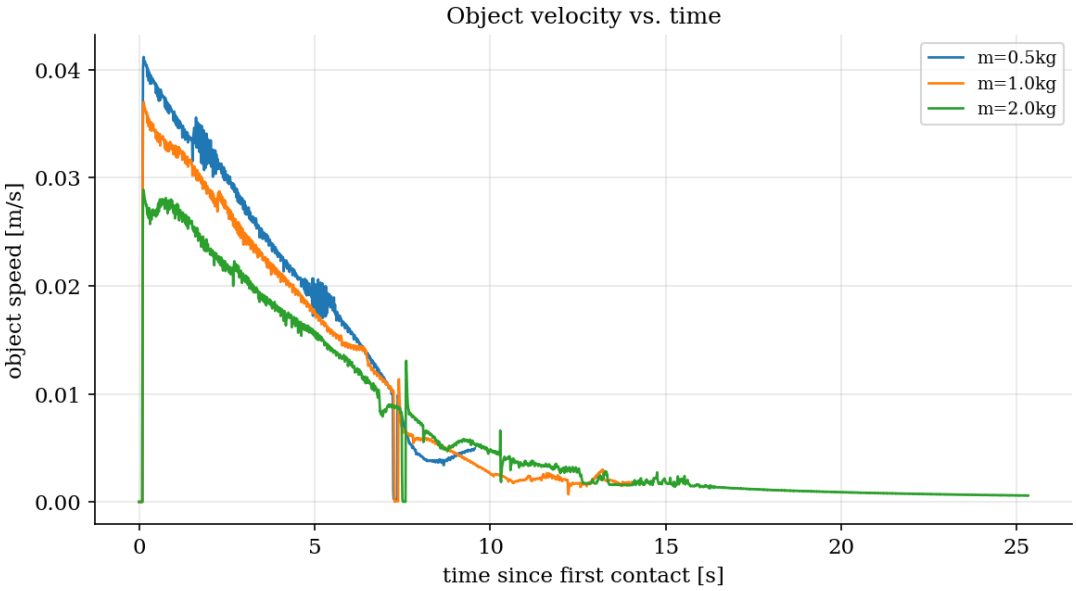
Figure 6.1a shows that the steady-state contact force scales with mass as predicted: the observed plateau is approximately 1.7-2.9 N for the 0.5 kg object, 3.5-4 N for the 1.0 kg object, and 7-8 N for the 2.0 kg object, closely tracking the theoretical values of 1.84 N, 3.68 N, and 7.36 N respectively (dotted lines). This confirms that the contact-force channel reproduces the analytically predicted force scale, and therefore that the force direction driving Eq. (5.6) is physically meaningful. All three runs additionally show a brief, shared drop in contact force at approximately the same point in the push (around 35–45 mm from the target), coinciding with a measurable reversal in the object's own yaw. This is related to the near-goal instability described in Section 6.4.1: as the object approaches the acceptance region, the desired direction $\hat{d}$ changes more rapidly, and even with the low-pass filter of (5.5) in place, a brief transient in contact force and object rotation remains. Notably, the amplitude of the yaw excursion decreases sharply with mass (approximately 34°, 20°, and 9° for the 0.5, 1.0, and 2.0 kg objects respectively), explained by the larger rotational inertia $I_{zz} = mc^2$ of the heavier objects resisting the same frictional torque more effectively.

The velocity profiles in Figure 6.1b show the same qualitative shape across all three masses with a brief transient followed by the commanded braking decay but the heavier objects converge more slowly toward the commanded push speed at a given time since contact, consistent with their larger inertia. The 0.5 kg object additionally exhibits a visible high-frequency oscillation in both its force and velocity traces (around 1-2 s and 5 s after contact) that is absent for the heavier objects; since the contact stiffness K_{normal} is held

fixed across all three runs, the natural frequency of the contact scales as $\sqrt{K_{\text{normal}}/m}$, making the lightest object more susceptible to contact chatter.



(a) Contact-force magnitude. The dotted lines indicate the theoretical steady-state values $F_{\text{theory}} = \mu_c mg$



(b) Object-speed magnitude during the same pushes.

Figure 6.1: Contact force and object velocity versus time since first contact for the cylinder at $\mu = 0.3$ and three object masses.

6.3 Systematic benchmark results

The full grid of 48 runs is summarised in Table 6.2, aggregated over the two initial positions. The complete per-run results appear in Appendix A.1. Because the two positions correspond to different push orientation, they are also reported separately in Table 6.3, which isolates the effect that Section 6.4.1 identifies as

the dominant driver of cube performance. The cylinder, having no discrete-face constraint, serves as the baseline for comparison throughout.

Table 6.2: Benchmark results over the full grid: success rate and mean final distance to the goal, per shape, object-side friction, and mass. Each cell aggregates the two initial positions A and B of Table 6.1.

Shape	μ	0.5 kg	1.0 kg	1.5 kg	2.0 kg
Cube	0.3	100 % / 25 mm	100 % / 25 mm	100 % / 25 mm	100 % / 25 mm
	0.5	50 % / 35 mm	50 % / 36 mm	100 % / 25 mm	0 % / 32 mm
	0.7	50 % / 34 mm	50 % / 33 mm	0 % / 31 mm	0 % / 42 mm
Cylinder	0.3	100 % / 25 mm	100 % / 25 mm	100 % / 25 mm	0 % / 29 mm
	0.5	100 % / 25 mm	100 % / 25 mm	0 % / 31 mm	0 % / 37 mm
	0.7	100 % / 25 mm	100 % / 25 mm	0 % / 34 mm	0 % / 45 mm

Each cell: success rate over the two positions / mean final distance. A successful run is censored at $d_{\text{done}} = 25$ mm by construction (see below), so 25 mm denotes a run that stopped where it was asked to.

The complete results of all 48 individual benchmark runs are reported in Appendix A, Table A.1.

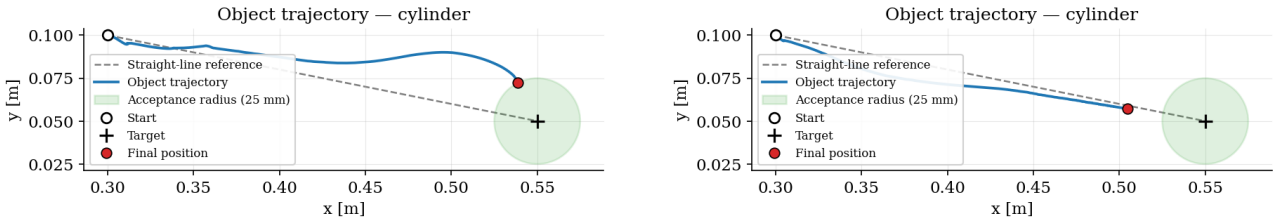
Table 6.3: Effect of initial obliquity, averaged over all masses and friction coefficients. Position A requires a push direction ~ 27° from the nearest cube face axis, position B only ~ 11°.

	Cube		Cylinder	
Metric	A (~ 27°)	B (~ 11°)	A	B
Success rate	5/12 (41 %)	9/12 (75 %)	7/12 (58 %)	7/12 (58 %)
Mean final distance [mm]	33.9	27.3	29.0	29.4
Mean lateral deviation [mm]	31.3	9.6	10.6	5.9
Max lateral deviation [mm]	72.9	34.7	27.6	14.1

Because the termination criterion and the success criterion share the same threshold d_{done}, the final distance of a successful run is censored at that value by construction: it indicates that the controller stopped where it was asked to, not how accurately it could have placed the object. Discrimination between successful runs therefore rests on completion time and lateral deviation rather than on final distance.

The contact-force channel is available for 97% of the push on average, ranging from 80% to 100% across the grid, the remainder being covered by the geometric fallback of Eq. (5.8). The controller therefore acts on directly measured contact force for nearly the entire interaction, the fallback intervening only during brief separations.

Cylinder: The cylinder succeeds in all 12 runs at 0.5 and 1.0 kg. At 1.5 kg it succeeds only at$\mu = 0.3$, and at 2.0 kg only at that same friction level, giving 14 successes out of 24 overall and 7/12 at each of the two positions. Performance is therefore governed by mass and friction, with no dependence on the initial obliquity. This follows from the mechanics of Section 4.1.5: because the contact normal is recomputed at every tick from the filtered desired direction, the push direction is never structurally misaligned with the goal, whatever the obliquity. The limiting factor is the force and torque demand of the heavy object. Figures 6.2 and 6.3 complement these numerical results with the corresponding cylinder trajectories.



(a) $m = 0.5\,\text{kg}$, $\mu = 0.3$.

(b) $m = 2.0\,\text{kg}$, $\mu = 0.7$.

Figure 6.2: Representative cylinder trajectories for a light, low-friction object and a heavy, high-friction object. The nominal case reaches the acceptance region, whereas the heavy-object run follows a more curved path and stops substantially farther from the target. Because both mass and friction change between panels, this figure provides a qualitative contrast between nominal and demanding conditions.

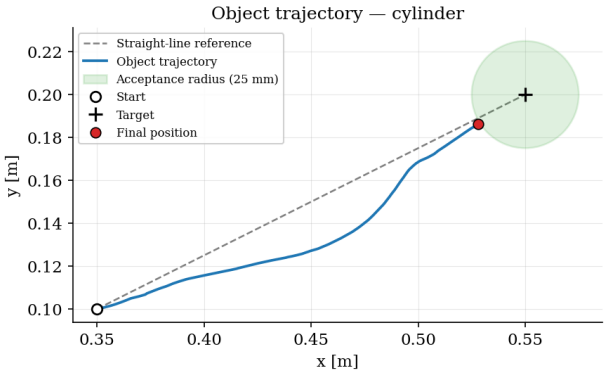


Figure 6.3: Representative trajectory of the 2.0 kg cylinder at $\mu = 0.3$. The trajectory approaches the acceptance region but terminates just outside the 25 mm success threshold, as reported in Table 6.2.

Cube: The cube succeeds in 14 of 24 runs, and its failures follow a different pattern. At $\mu = 0.3$, it succeeds in all eight runs, across every mass and both positions. Failures appear only at $\mu \geq 0.5$, and at 0.5 and 1.0 kg they occur exclusively at position A, the high-obliquity geometry. Accuracy is also not monotonic in mass: at $\mu = 0.5$ and position A, the maximum lateral deviation falls steadily from 78.6 mm at 0.5 kg to 65.4 mm at 2.0 kg, and the cube succeeds at both positions at 1.5 kg before failing before failing at both positions at 2.0 kg. A heavier object resists the parasitic torque of Eq. (4.7) better, since the inertia $I_{zz} = mc^2 grows with mass while the off-centre normal force does not; the two effects act in opposite directions, and$

6.4 Failure modes

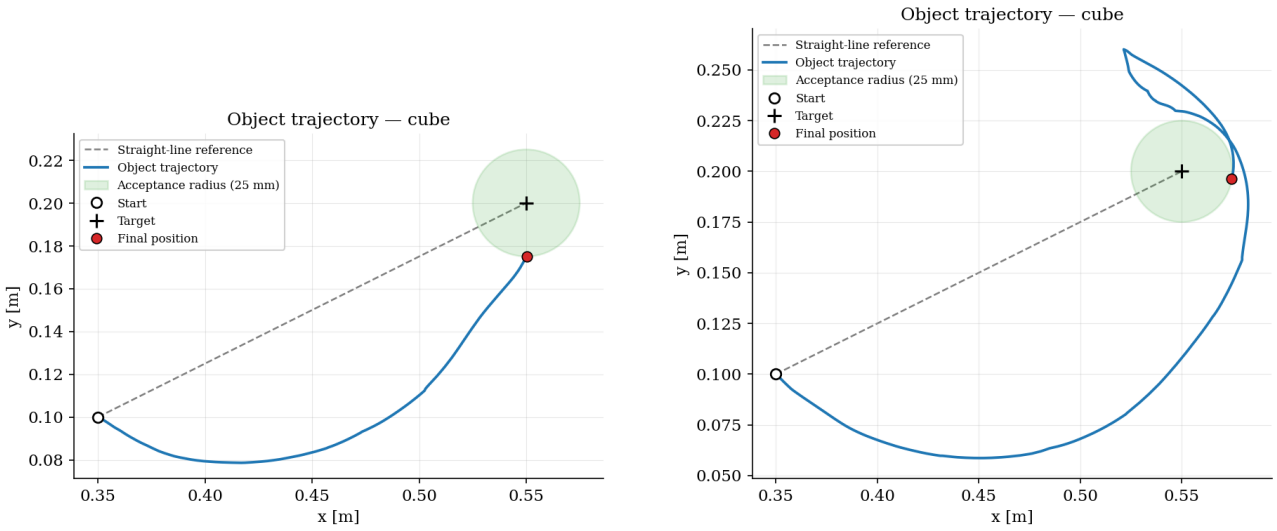
The torque bounds of Section 4.2.1 place a hard limit on what the arm can deliver against a heavy, high-friction object. This section examines the two failure modes actually observed in the campaign of Section 6.3, the near-goal directional instability and the divergence it can trigger, and documents the fix each motivated.

6.4.1 Instability near the goal

Mechanism. Figures 6.4a and 6.5 compare the same object (0.5 kg, $\mu = 0.3$) pushed from position A ($\sim 27°$ obliquity) and position B ($\sim 11°$). Both trajectories show the same qualitative signature: the object is first displaced almost purely along the x-axis, with little or no progress toward the goal along y, before correcting toward the target only in the second half of the push. This two-phase, "dog-leg" shape is a direct consequence of the force-feedback correction of (5.6). At the start of contact on a flat face, the measured contact force is nearly normal to that face, i.e. aligned with the frozen face axis rather than with the instantaneous goal direction θ_d. The resulting angular deviation $\delta_f = \theta_f - \theta_d$ is therefore approximately equal in magnitude to the obliquity itself, and the correction term $(K_F + 1)\,\delta_f$ actively pulls the commanded direction *away* from the goal and toward the face normal. Only once sliding or rotation has built up a measurable tangential force component does θ_f move enough for the correction to stop opposing progress along y, by which point the object has typically already accumulated a substantial, unwanted yaw. This is the opposite of the cylinder's behaviour (Section 6.3), where the tracked contact normal is never structurally misaligned with the goal in the first place, so the same force-feedback law does not fight the desired direction.

This is a direct consequence of combining the reactive force-feedback law of (5.6) with the cube's frozen discrete face axis: the two design choices are individually well-justified, but their interaction degrades sharply once the required push direction exceeds roughly 30° from the nearest face axis, consistent with the failure pattern observed across the benchmark grid.

Friction analysis The dog-leg shape is present at both friction levels, but its consequences differ sharply. Comparing Figures 6.4a and 6.4b (same mass and position, $\mu = 0.3$ vs. $\mu = 0.7$) shows that the higher-friction case accumulates substantially more unwanted rotation and, in this instance, diverges(Section 6.4.2). This follows directly from the torque relation of (4.7), $\tau_o \approx -p_y f_n$: the normal force f_n required merely to overcome table friction scales with the Coulomb prediction $F_{\text{theory}} = \mu_c\, m\, g$, where μ_c is the harmonic-mean friction combining the object and table coefficients. For the 0.5 kg case, $\mu_c(0.3) \approx 0.375$ gives $F_{\text{theory}} \approx 1.84$ N, while $\mu_c(0.7) \approx 0.583$ gives $F_{\text{theory}} \approx 2.86$ N, a 55% increase. Since the parasitic torque scales directly with f_n for a given lateral offset p_y, the same imperfection in lateral centering produces proportionally more unwanted rotation at higher friction, independently of any change in obliquity. A secondary effect compounds this: a higher object-side μ also raises the pusher-object friction μ_p of the friction cone ((4.20)), allowing a larger tangential force f_t before slipping. The full torque expression of (4.6), $\tau_o = a f_t - p_y f_n$, retains only the second term under the assumption that f_t stays small (Section 4.1.4); at $\mu = 0.7$ this assumption holds less well, and the neglected $a f_t$ term becomes a second, uncorrected source of rotation. Both effects push in the same direction, which follows from the qualitative ranking observed across all tested combinations: performance degrades with obliquity, and independently degrades further with friction, with the two effects compounding at position A/B and $\mu = 0.7$.



(a) Position A (~ 27°), $\mu = 0.3$.

(b) Position A (~ 27°), $\mu = 0.7$.

Figure 6.4: Effect of friction on the cube trajectory for the same mass and initial geometry (0.5 kg, position A): $\mu = 0.3$ (left) and $\mu = 0.7$ (right). Higher friction produces greater curvature and slower convergence toward the acceptance region.

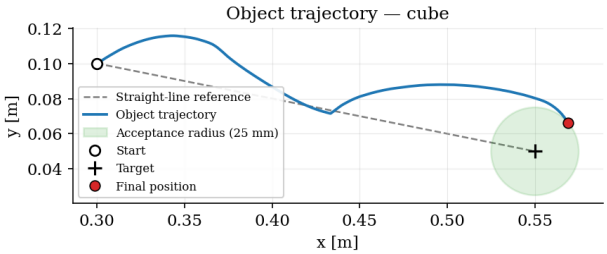


Figure 6.5: Cube push at position B (~ 11° obliquity), with $m = 0.5$ kg and $\mu = 0.3$. Compared with the position A case in Figure 6.4a, the lower-obliquity trajectory requires less correction before approaching the target.

6.4.2 Stall / regression

The interaction described above can, in the worst case, produce a run that does not merely fail to converge but actively diverges: once the object overshoots close to the goal and the desired direction reverses, the speed profile of Eq. (5.12) reacts only to the magnitude of the remaining distance, not to the sign of progress. As the object moves away and the distance grows again, the profile ramps back toward full V_{push} exactly as if a new push were beginning, rather than signalling a regression. This behaviour was also observed under camera-based perception in an earlier campaign (0.5 kg cube, $\mu = 0.7$: final distance 52.45 mm), before the safety-net described below was wired in.

Figure 6.6 shows this effect for the 0.5 kg cube with $\mu = 0.7$ at position A, with the stall/regression safety-net disabled. The object initially moves toward the target and loops around its neighbourhood, but the controller does not recognise that subsequent motion is increasing the goal distance. The velocity profile therefore rises again and continues to drive the object away, ending approximately 890 mm from the target.

The stall-detection mechanism is intended for exactly this situation. It tracks the best distance achieved

and triggers a reposition when no measurable progress occurs for 4 s; if the object is already within $2 \times d_{\text{done}}$, completion can instead be accepted consistently with the tolerance defined in Section 6.1. With the safety net enabled, the identically configured run terminates after 13.6 s at 43.0 mm from the goal (Appendix A.1). The run still misses the acceptance region, since position A at $\mu = 0.7$ is one of the configurations for which the cube fails under perfect perception, but the divergence is removed: the final error drops from approximately 890 mm to 43 mm. The safety net does not make a hard configuration succeed; it prevents an unrecoverable one.

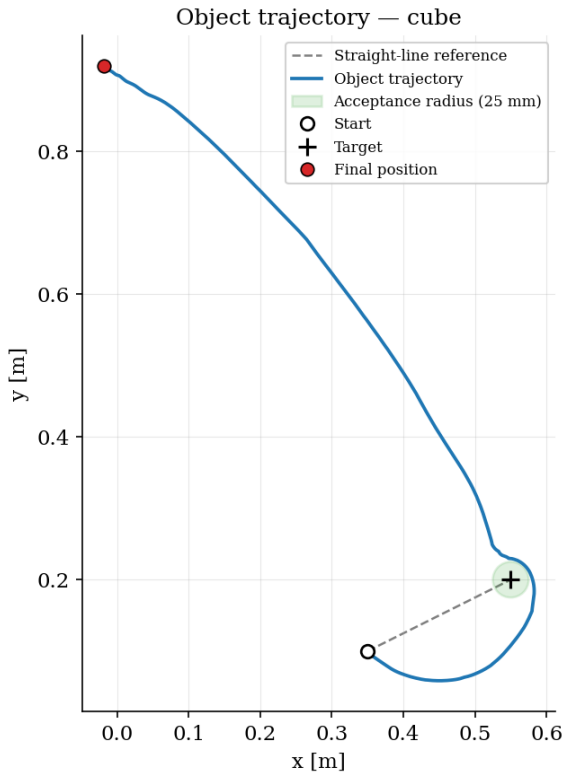


Figure 6.6: Divergent trajectory of the 0.5 kg cube with $\mu = 0.7$ at position A when the stall/regression safety-net is disabled. The object first approaches and passes around the target neighbourhood, then moves progressively farther away because renewed growth of the goal distance is interpreted by the speed profile as a reason to accelerate.

6.5 Lateral centering contribution

To isolate the individual contribution of the two corrective terms of Sections 5.2.4 and 5.2.1, the lateral centering gain K_{lat} (Eq. (5.10)) and the direction low-pass filter $D_{\hat{d}}$ (Eq. (5.5)) were disabled independently on a 0.25 kg object at $\mu = 0.3$. This object is lighter than any object in the benchmark grid, so the rotational drift of Section 6.4.1 is more pronounced. The two ablation geometries are positions C, with approximately 14° obliquity, and D, with approximately 37° obliquity, of Table 6.1. Both are evaluated using the ground-truth object position, so that the comparison isolates the controller from any perception effect.

For the cube, every condition succeeds at position C and fails at position D. The cube success rate is therefore 50 % for all three conditions and carries no discriminating information. The cylinder follows a

different position-dependent pattern. Table 6.4 therefore reports the final distance separately for positions C and D.

Table 6.4: Controller-component ablation: final distance to the goal [mm] for a 0.25 kg object at $\mu = 0.3$, reported separately for positions C and D of Table 6.1. Values at or below the 25 mm threshold correspond to successful runs.

	Cube		Cylinder	
Condition	C	D	C	D
Full controller (baseline)	25	39	45	25
No direction filter ($D_{\hat{d}} = 1$)	25	38	42	25
No lateral centering ($K_{\text{lat}} = 0$)	25	**107**	25	31

At the low-obliquity position C, the three cube conditions are indistinguishable. All reach the acceptance region and are censored at $d_{\text{done}} = 25$ mm. Neither corrective term is needed when the required cube push direction is already close to a face axis.

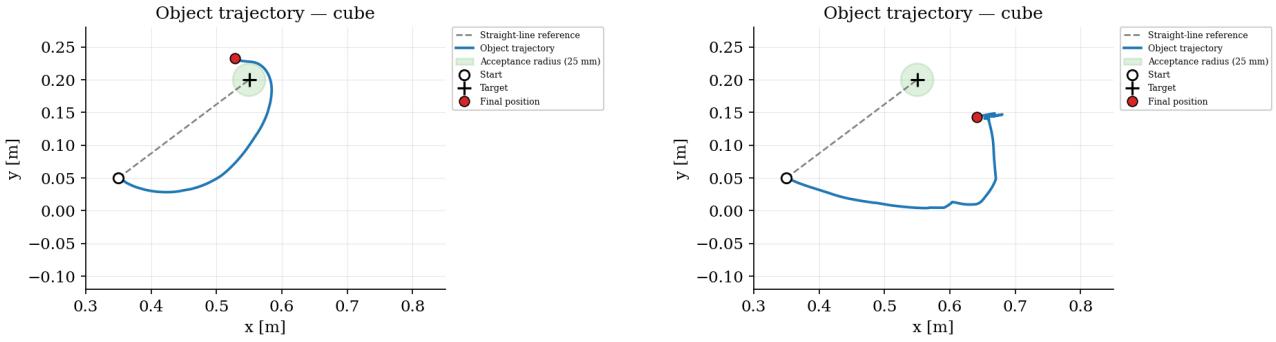
At the high-obliquity position D, the cube separates the two terms clearly. Removing the direction filter changes the final distance only from 39 to 38 mm, a difference too small to interpret from one run per cell. Removing lateral centering degrades the final distance from 39 to 107 mm, a factor of approximately 2.7, and produces the trajectory of Figure 6.7b. The object is driven almost purely along x, misses the goal, and turns back toward it only after the pusher has travelled past the target neighbourhood. This is the mechanism of Section 6.4.1 with its main corrective channel removed. Lateral centering keeps the contact point aligned on the pushing face while the force-feedback correction opposes the initial obliquity-driven misalignment. Without it, the early drift is not corrected, and the frozen cube-face axis leaves the controller no other way to recover.

The baseline itself does not succeed at position D, finishing 39 mm from the goal against a 25 mm threshold. The ablation geometries were deliberately chosen at the difficult end of the obliquity range, where the contribution of each term is most visible. The comparison is therefore between a near miss and an outright miss, rather than between a success and a failure.

The cylinder shows no comparable systematic dependence. Averaged over positions C and D, removing lateral centering reduces its mean final distance from approximately 35 to 28 mm. However, the position-specific effects are opposite: position C improves from 45 to 25 mm, whereas position D degrades from 25 to 31 mm. With only two runs per condition, this average difference is not meaningful. This weak and geometry-dependent response is consistent with the cylinder's axisymmetry: its contact point is fixed at radius R by geometry, so it does not exhibit the same face-relative lateral offset as the cube.

The absence of a measurable direction-filter effect also requires qualification. The filter acts on the rate of rotation of $\hat{d}$, and these ablation runs do not produce the rapid near-goal reversal that the filter was introduced to suppress. The filtered and unfiltered directions therefore remain similar throughout these trials. The absence of an effect in this ablation indicates that the test does not strongly excite the near-goal

reversal regime of Section 6.4.1; it does not establish that the direction filter is unnecessary.



(a) Full controller.

(b) No lateral centering.

Figure 6.7: Cube trajectories for the high-obliquity ablation case (0.25 kg, $\mu = 0.3$, position D of Table 6.1): full controller (left) and lateral centering disabled (right). Removing lateral centering causes the trajectory to diverge from the goal.

6.6 Robustness to perception

The benchmark of Section 6.3 supplies the controller with the object's exact position at every control tick. This section relaxes that assumption in three stages, using the camera model of Section 5.3 (30 Hz, 80 ms latency, $\sigma = 3$ mm): intermittent occlusion during contact, at frame loss probabilities of 50 % and 90 %, and the limiting case in which the object is localised once and never observed again, so that the controller falls back on the dead reckoning criterion of Eq. (5.17).

All three levels are evaluated on the same eight configurations: both shapes, 0.5 and 1.0 kg, $\mu = 0.3$, positions A and B of Table 6.1. These eight were selected because the controller succeeds in every one of them under ground truth, so that any degradation observed here is attributable to the perception channel alone.

6.6.1 Occlusion and frozen camera

The three regimes are separated by two orders of magnitude in perception error (Table 6.5). Losing half the camera frames leaves a mean error of 4.0 mm between the perceived and the true object position; losing nine frames out of ten leaves 6.6 mm. Only when the estimate is frozen does the error reach 199.6 mm, since the object travels 200 to 250 mm while the belief remains at its initial value.

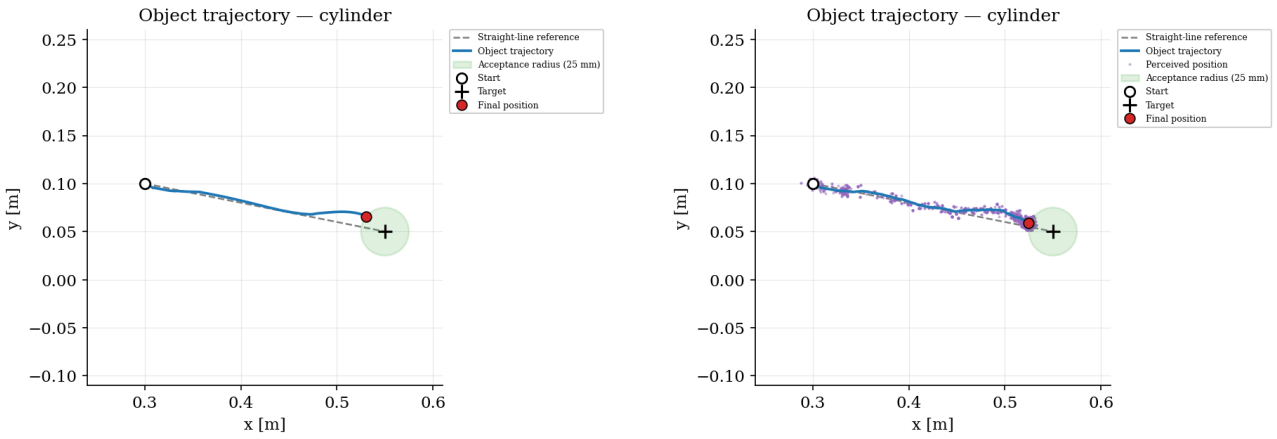
This ordering follows from a simple bound. The error accumulated between two successful observations scales as $v\,\Delta t$, where v is the object speed and Δt the interval since the last valid frame. At the 60 mm/s push speed of Table 5.1 and a nominal 30 Hz frame rate, a frame loss probability p gives $\Delta t \approx (1/30)/(1-p)$, so that $p = 0.9$ still leaves $\Delta t \approx 0.33$ s and an error of roughly 20 mm at worst. The frozen case corresponds to $p = 1$, where the bound diverges.

The distinction matters for the practical relevance of the experiment. A real RGB-D pipeline, partially occluded by the manipulator during contact, operates in the intermittent regime. Permanent and complete

loss of tracking is a singular case, not the expected behaviour of a camera whose view is obstructed for part of the push.

6.6.2 Trajectory analysis

At 50 % frame loss the mean lateral deviation from the straight line reference is 15.8 mm, identical to the 15.8 mm measured under ground truth. The path the object follows is indistinguishable from the nominal one, as Figure 6.8 shows. Yet none of the eight runs is scored a success: all eight stop between 26.9 and 32.1 mm from the goal, a narrow band just outside the 25 mm acceptance radius.



(a) Ground truth, 25.0 mm.

(b) 50 % frame loss, 27.1 mm.

Figure 6.8: Effect of 50 % frame loss on a nominal configuration (cylinder, 1.0 kg, $\mu = 0.3$, position B). The pushed trajectory follows the same path in both cases; averaged over the eight configurations, the mean lateral deviation is 15.8 mm with and without occlusion. Only the stopping point differs: the occluded run terminates 2.1 mm outside the acceptance radius because the termination test is evaluated on the perceived position, shown in purple, which carries a mean error of 4.0 mm.

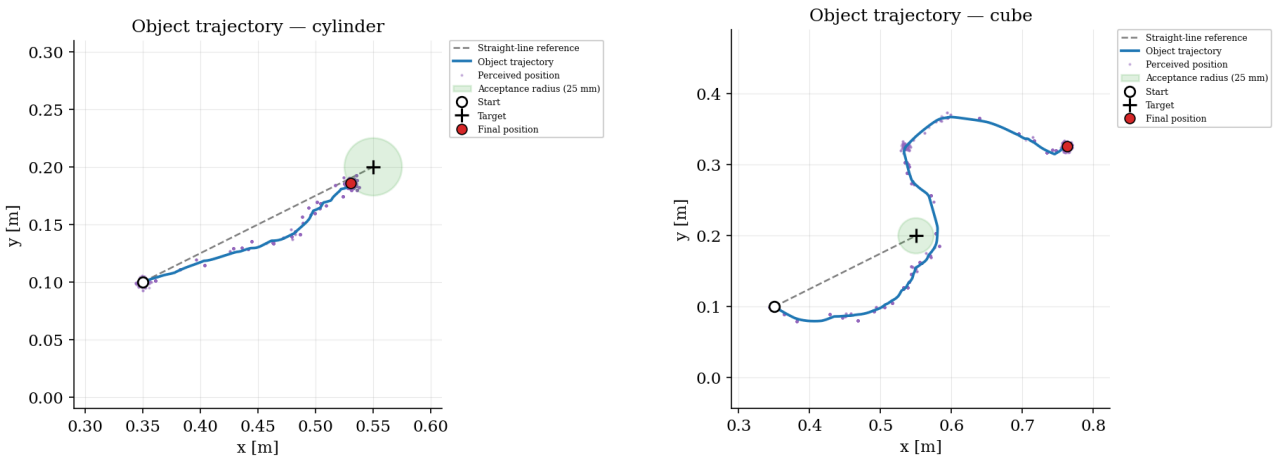
The explanation is in the termination criterion, not in the control law. The primary completion test of Section 5.2 compares the *perceived* distance to the goal against d_{done}. With a perception error of 4 mm, that distance can read below 25 mm while the true distance is 28 mm, and the controller stops in the belief that it has arrived. The terminal error is therefore the acceptance threshold plus the perception error, which is what the measurements give: $25 + 4 \approx 28.4$ mm observed on average.

Under degraded perception the controller does not push less accurately. It stops less accurately. The binary success rate cannot express that distinction, which is why Table 6.5 reports the lateral deviation and the median final distance alongside it.

Four of the eight runs succeed at 90 % occlusion and the median final distance is 25.8 mm, marginally better than at 50 %. A staler estimate makes the braking profile of Eq. (5.12) over-estimate the remaining distance, so the object is carried slightly further before the controller stops.

The mean, however, rises to 59.5 mm. It is dominated by two cube runs at position A, which finish 82 and 248 mm from the goal; the remaining six runs average 24.3 mm. Those two are the high obliquity cube configurations, where the frozen face axis of Section 5.2.2 is already the limiting factor under perfect per-

ception (Section 6.4.1). Stale perception and initial obliquity compound: each is survivable alone, and their combination is not. Figure 6.9 contrasts the two shapes at identical mass, friction, position and occlusion level.



(a) Cylinder, 24.4 mm.

(b) Cube, 247.9 mm.

Figure 6.9: Shape asymmetry under 90 % frame loss at high obliquity (1.0 kg, $\mu = 0.3$, position A). The cylinder reaches the acceptance region, while the cube diverges to 248 mm. The continuously re-evaluated contact normal of Section 4.1.5 keeps the cylinder's push direction aligned with the goal whatever the obliquity, whereas the cube's frozen face axis offers no mechanism to recover once the early drift has gone uncorrected.

Mean and median must be read together here. The success rate alone would suggest, misleadingly, that 90 % occlusion is more benign than 50 %.

6.6.3 Limits

When the estimate is never refreshed, one of the eight runs succeeds and the mean final distance reaches 45.9 mm. The controller continues along the direction planned from the initial capture, and the dead reckoning criterion of Eq. (5.17) terminates the push on the pusher's own displacement. That criterion keeps the motion bounded, and no run diverges, but it cannot correct the positional error accumulated in the meantime.

One effect works in the opposite direction. Because dead reckoning does not test the perceived distance, the final distance of a completed run is no longer censored at d_{done}, and the successful run finishes at 22.4 mm instead of stopping at the threshold.

6.6.4 Summary

Three results follow from this campaign. First, the controller tolerates the loss of most visual observations without any degradation of the pushed trajectory: at 50 % frame loss the lateral deviation is exactly the ground truth value. Second, the residual failures at that level are an artefact of evaluating the stopping condition on a noisy measurement, and not a loss of control authority; widening d_{done} by the expected

Table 6.5: Effect of degraded perception on the eight configurations common to all campaigns (cube and cylinder, 0.5 and 1.0 kg, $\mu = 0.3$, positions A and B). Perception error is the mean distance between the perceived and the true object position over the push. Mean and median final distance diverge at 90 % occlusion because two high obliquity cube runs dominate the mean.

	Ground truth	50 % occl.	90 % occl.	Frozen
Success rate	8/8	0/8	4/8	1/8
Mean final distance [mm]	25.0	28.4	59.5	45.9
Median final distance [mm]	25.0	28.1	25.8	42.7
Mean completion time [s]	15.5	16.4	19.4	9.0
Mean perception error [mm]	0.0	4.0	6.6	199.6
Max perception error [mm]	0	16	99	259
Mean lateral deviation [mm]	15.8	15.8	28.3	38.0
Sensor availability [%]	93	83	90	90

perception error, or requiring the condition to hold over several consecutive frames, would recover most of them without touching the control law. Third, what the controller does not tolerate is the combination of stale perception with high initial obliquity on a flat faced object, which is the same coupling identified in Section 6.4.1 and is not specific to the perception channel.

7 Conclusion

7.1 Summary

This thesis investigated whether goal-directed planar pushing can be performed reliably by a single robotic arm fitted with a spherical end-effector, using the direction of the measured contact force as the sole feedback signal about the interaction, without any model of the object's dynamics. A reactive, impedance-based controller was designed around this estimate, implemented as a finite-state machine (approach, push, reposition, done) and augmented with two corrective terms,lateral centering and direction filtering, together with a camera-based perception model and a stall/regression safety net. The approach was validated in a Drake-based physics simulation across a benchmark grid spanning object shape, mass, friction, and initial contact obliquity.

7.2 Main findings

Several conclusions can be drawn from the results presented in Chapter 6.

First, the controller converges reliably to the target region across most of the tested grid, with the main failure modes concentrated in high-obliquity, high-friction configurations, where the parasitic torque of Eq. (4.6) is largest (Section 6.4).

Second, the ablation study of Section 6.5 shows that the individual controller components do not contribute equally across object shapes. Disabling lateral centering degrades the cube's final position error from 39 mm to 107 mm, confirming that this correction is necessary to correct the orientation-dependent contact drift specific to a flat-faced object. The same ablation produces a much smaller effect for the cylinder (35 mm vs. 30 mm), consistent with its axisymmetric geometry, which does not generate the same lateral drift. Direction filtering, by contrast, produced no measurable difference on the tested configuration (Table 6.4). Direction filtering produced no measurable difference here, though the ablation configurations do not enter the near-goal regime the filter was designed for.

Third, the stall/regression safety net addresses a real and previously undetected failure mode of the controller: without it, a push that overshoots near the goal can diverge indefinitely, because the velocity profile

reacts only to the magnitude of the remaining distance and not to the sign of progress (Section 6.4.2). Under an identical configuration, enabling the safety net converts this divergent run into a better completion.

Fourth, the perception experiments of Section 6.6 separate two effects that a binary success rate conflates. Losing half or even nine tenths of the camera frames leaves the pushed trajectory unchanged, the mean lateral deviation at 50% frame loss is 15.8 mm, exactly the ground-truth value, while shifting the stopping point by the magnitude of the perception error, since the termination test is evaluated on the perceived position. Only when the estimate is frozen for the entire push does performance collapse, and even then the dead-reckoning fallback keeps every run bounded. The controller therefore does not require continuous tracking to push accurately; it requires it to stop accurately.

7.3 Contributions

The main contributions of this thesis can be summarised as follows.

1. An adaptation of reactive force-feedback pushing control, previously validated on an omnidirectional mobile base, whose end-effector can translate freely in the plane, to an arm-mounted spherical pusher on a 7-DoF manipulator, where the reachable workspace, the joint torque limits, and the need for an explicit approach and repositioning phase all constrain the push. The adaptation includes a shape-dependent contact normal that has no equivalent in the original formulation

2. A finite-state reactive controller combining this estimate with lateral-centering and direction-filtering corrections, together with a stall/regression safety net that recovers from an otherwise unrecoverable divergence mode.

3. A camera perception model (latency, noise, and contact-induced occlusion) with a dead-reckoning fallback, allowing the controller to complete a push without continuous object tracking.

4. A systematic benchmark and ablation study, in simulation, quantifying the individual contribution of each controller component across object shape, mass, friction, and obliquity, and showing that this contribution is shape-dependent.

7.4 Limitations

The validation presented in this thesis is entirely in simulation. While Drake's contact model is physically grounded, it does not capture every aspect of a real robot: actuator dynamics, force estimation from actual joint torques rather than an idealised penetration measurement, real camera hardware, and unmodelled compliance in the arm and end-effector could all affect the results reported here. The benchmark grid, while covering a meaningful range of mass, friction, and obliquity, was limited to two object shapes (cube and cylinder) and to quasi-static pushing; the controller's behaviour under faster pushes, non-convex objects,

or cluttered environments with multiple obstacles remains untested. Finally, the camera perception model used here, though intended to be representative of a real RGB-D pipeline's error characteristics, is itself simulatedk.

7.5 Future work

Several directions follow naturally from these limitations. The most immediate is hardware validation on a physical arm, where the contact-force channel assumed here would be supplied by real hardware,a wrist force/torque sensor or a tactile fingertip. Integrating a real-time RGB-D perception pipeline in place of the simulated camera model would test whether the robustness observed here under simulated noise and occlusion transfers to real sensing conditions. The benchmark could be extended to additional object shapes, including non-convex and irregular geometries, and to scenarios involving multiple objects or obstacles. Finally, the scenario-dependent effect of direction filtering observed during this work suggests a concrete open question: characterising, analytically or empirically, the class of push trajectories for which this term is beneficial versus counterproductive, so that it could eventually be applied adaptively.

A

Complete benchmark results

Table A.1: Complete benchmark: all 48 individual runs, using the ground-truth (Drake) object position.

Shape	Pos.	m [kg]	μ	Success	Time [s]	Final [mm]	Lat. max [mm]	Sensor [%]
Cube	A	0.5	0.3	Yes	12.21	25.0	61.5	97.2
Cube	B	0.5	0.3	Yes	14.98	24.9	30.5	94.7
Cube	A	0.5	0.5	No	–	45.5	78.6	90.5
Cube	B	0.5	0.5	Yes	15.32	24.9	36.6	93.4
Cube	A	0.5	0.7	No	–	43.0	95.9	98.1
Cube	B	0.5	0.7	Yes	17.08	25.0	40.9	97.0
Cube	A	1.0	0.3	Yes	17.50	25.0	60.6	98.6
Cube	B	1.0	0.3	Yes	20.03	25.0	25.2	98.4
Cube	A	1.0	0.5	No	–	46.1	75.4	98.9
Cube	B	1.0	0.5	Yes	24.66	25.0	36.9	98.9
Cube	A	1.0	0.7	No	–	40.9	90.7	98.4
Cube	B	1.0	0.7	Yes	29.12	25.0	47.2	99.2
Cube	A	1.5	0.3	Yes	25.46	25.0	57.8	99.2
Cube	B	1.5	0.3	Yes	26.20	25.0	25.6	99.0
Cube	A	1.5	0.5	Yes	39.71	25.0	69.9	99.4
Cube	B	1.5	0.5	Yes	35.82	25.0	36.4	99.4
Cube	A	1.5	0.7	No	–	32.6	85.6	99.5
Cube	B	1.5	0.7	No	–	28.8	42.6	99.4
Cube	A	2.0	0.3	Yes	34.39	25.0	53.9	99.4
Cube	B	2.0	0.3	Yes	34.10	25.0	27.1	99.2
Cube	A	2.0	0.5	No	–	30.8	65.4	99.5
Cube	B	2.0	0.5	No	–	32.3	33.6	99.5
Cube	A	2.0	0.7	No	–	42.3	80.1	99.5
Cube	B	2.0	0.7	No	–	42.2	34.1	99.4
Cylinder	A	0.5	0.3	Yes	10.80	25.0	26.0	81.3
Cylinder	B	0.5	0.3	Yes	13.75	25.0	29.8	80.3
Cylinder	A	0.5	0.5	Yes	12.49	25.0	35.6	93.2

Continued on the next page

Table A.1 continued from the previous page

Shape	Pos.	m [kg]	μ	Success	Time [s]	Final [mm]	Lat. max [mm]	Sensor [%]
Cylinder	B	0.5	0.5	Yes	14.40	25.0	19.9	87.4
Cylinder	A	0.5	0.7	Yes	13.23	25.0	40.9	95.3
Cylinder	B	0.5	0.7	Yes	17.42	25.0	15.7	92.8
Cylinder	A	1.0	0.3	Yes	15.97	25.0	21.2	98.6
Cylinder	B	1.0	0.3	Yes	18.38	25.0	13.1	94.8
Cylinder	A	1.0	0.5	Yes	22.46	25.0	30.4	98.8
Cylinder	B	1.0	0.5	Yes	25.24	25.0	12.3	93.4
Cylinder	A	1.0	0.7	Yes	29.17	25.0	33.3	99.3
Cylinder	B	1.0	0.7	Yes	32.54	25.0	10.6	98.6
Cylinder	A	1.5	0.3	Yes	26.75	25.0	23.2	99.3
Cylinder	B	1.5	0.3	Yes	28.80	25.0	10.5	97.4
Cylinder	A	1.5	0.5	No	–	31.0	26.1	99.4
Cylinder	B	1.5	0.5	No	–	30.8	14.1	98.8
Cylinder	A	1.5	0.7	No	–	33.3	27.7	99.5
Cylinder	B	1.5	0.7	No	–	33.9	13.0	99.0
Cylinder	A	2.0	0.3	No	–	29.0	21.4	99.4
Cylinder	B	2.0	0.3	No	–	29.3	8.3	98.5
Cylinder	A	2.0	0.5	No	–	35.9	22.4	99.6
Cylinder	B	2.0	0.5	No	–	38.4	11.7	99.1
Cylinder	A	2.0	0.7	No	–	44.0	23.0	99.6
Cylinder	B	2.0	0.7	No	–	45.8	10.6	99.1

Completion time is reported only for runs meeting the 25 mm success criterion. “Sensor” is the fraction of the push over which the contact-force channel was available, the remainder being covered by the geometric fallback of Eq. (5.8).

Bibliography

[1] Russ Tedrake and the Drake Development Team. *Drake: Model-Based Design and Verification for Robotics*. Online. Available at https://drake.mit.edu. 2019.

[2] RoboDK. *Franka Emika Panda Robot*. RoboDK. URL: https://robodk.com/robot/Franka/Emika-Panda (visited on 08/07/2026).

[3] Ivan Mehta and Rebecca Bellan. *Runway Releases Its First World Model, Adds Native Audio to Latest Video Model*. TechCrunch, republished by Yahoo Tech. Dec. 11, 2025. URL: https://tech.yahoo.com/ai/meta-ai/articles/runway-releases-first-world-model-170000394.html (visited on 08/06/2026).

[4] Matthew T. Mason. "Mechanics and Planning of Manipulator Pushing Operations". In: *The International Journal of Robotics Research* 5.3 (1986), pp. 53–71. DOI: 10.1177/027836498600500303.

[5] Kevin M. Lynch and Matthew T. Mason. "Stable Pushing: Mechanics, Controllability, and Planning". In: *The International Journal of Robotics Research* 15.6 (1996), pp. 533–556. DOI: 10.1177/027836499601500602.

[6] Suresh Goyal, Andy Ruina, and Jim Papadopoulos. "Planar Sliding with Dry Friction. Part 1: Limit Surface and Moment Function". In: *Wear* 143.2 (1991), pp. 307–330.

[7] Michael Posa, Cecilia Cantu, and Russ Tedrake. "A Direct Method for Trajectory Optimization of Rigid Bodies Through Contact". In: *The International Journal of Robotics Research* 33.1 (2014), pp. 69–81. DOI: 10.1177/0278364913506757.

[8] Mehmet R. Dogar and Siddhartha S. Srinivasa. "A Framework for Push-Grasping in Clutter". In: *Robotics: Science and Systems (RSS)*. 2011. DOI: 10.15607/RSS.2011.VII.009.

[9] Francois R. Hogan and Alberto Rodriguez. "Reactive Planar Non-Prehensile Manipulation with Hybrid Model Predictive Control". In: *The International Journal of Robotics Research* 39.7 (2020), pp. 755–773.

[10] Chelsea Finn and Sergey Levine. “Deep Visual Foresight for Planning Robot Motion”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:1610.00696. 2017. DOI: 10.1109/ICRA.2017.7989324.

[11] John Lloyd and Nathan F. Lepora. “Goal-Driven Robotic Pushing Using Tactile and Proprioceptive Feedback”. In: *IEEE Robotics and Automation Letters* 6.2 (2021). arXiv:2012.01859, pp. 2176–2183.

[12] Kevin M. Lynch, Hajime Maekawa, and Kazuo Tanie. “Manipulation and Active Sensing by Pushing Using Tactile Feedback”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 1992, pp. 416–421. DOI: 10.1109/IROS.1992.587370.

[13] Gaotian Wang, Kejia Ren, and Kaiyu Hang. “UNO Push: Unified Nonprehensile Object Pushing via Non-Parametric Estimation and Model Predictive Control”. In: *arXiv preprint arXiv:2403.13274* (2024). URL: https://arxiv.org/abs/2403.13274.

[14] Nils Dengler, David Großklaus, and Maren Bennewitz. “Learning Model-Based Control for Non-Prehensile Manipulation”. In: *arXiv preprint arXiv:2203.02389* (2023). URL: https://arxiv.org/abs/2203.02389.

[15] Sebastian Dengler, Tim Welschehold, and Oliver Brock. “Deep Reinforcement Learning for Contact-Rich Manipulation”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2022, pp. 10473–10480.

[16] Yuzhe Qin et al. “A Universal Pushing Controller for Object Delivery Using Vision-Based Reinforcement Learning”. In: *arXiv preprint* (2024).

[17] Neville Hogan. “Impedance Control: An Approach to Manipulation”. In: *Journal of Dynamic Systems, Measurement, and Control* 107.1 (1985), pp. 1–7. DOI: 10.1115/1.3140702.

[18] Luigi Villani and Joris De Schutter. “Force Control”. In: *Springer Handbook of Robotics*. Springer, 2008, pp. 161–185. DOI: 10.1007/978-3-540-30301-5_8.

[19] Adam Heins and Angela P. Schoellig. *Force Push: Robust Single-Point Pushing with Force Feedback*. IEEE Robotics and Automation Letters 9(8):6856–6863, 2024. 2024. URL: https://arxiv.org/pdf/2401.17517.

[20] Alessandro De Luca et al. “Collision Detection and Safe Reaction with the DLR-III Lightweight Manipulator Arm”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2006, pp. 1623–1630. DOI: 10.1109/IROS.2006.282053.

[21] Lucas Manuelli and Russ Tedrake. “Localizing External Contact Using Proprioceptive Sensors: The Contact Particle Filter”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2016, pp. 5062–5069. DOI: 10.1109/IROS.2016.7759743.

[22] Paul J. Besl and Neil D. McKay. “A Method for Registration of 3-D Shapes”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14.2 (1992), pp. 239–256. DOI: 10.1109/34.121791.

[23] Stefan Hinterstoisser et al. “Model Based Training, Detection and Pose Estimation of Texture-Less 3D Objects in Heavily Cluttered Scenes”. In: *Asian Conference on Computer Vision (ACCV)*. Vol. 7724. 2012. DOI: 10.1007/978-3-642-33885-4_60.

[24] Yu Xiang et al. “PoseCNN: A Convolutional Neural Network for 6D Object Pose Estimation in Cluttered Scenes”. In: *Robotics: Science and Systems (RSS)*. arXiv:1711.00199. 2018. DOI: 10.15607/RSS.2018.XIV.019.

[25] Chen Wang et al. “DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019), pp. 3343–3352. DOI: 10.1109/CVPR.2019.00346.

[26] Jeannette Bohg et al. “Interactive Perception: Leveraging Action in Perception and Perception in Action”. In: *IEEE Transactions on Robotics* 33.6 (2017). arXiv:1604.03670, pp. 1273–1291. DOI: 10.1109/TRO.2017.2721939.